

Spatial Modeling Algorithms for Reaction and Transport (SMART) in Realistic Cell Geometries

Emmet A. Francis, Justin Laughlin, Jørgen S. Dokken, Henrik Finsberg,
Christopher T. Lee, Marie E. Rognes, Padmini Rangamani



$\neq$

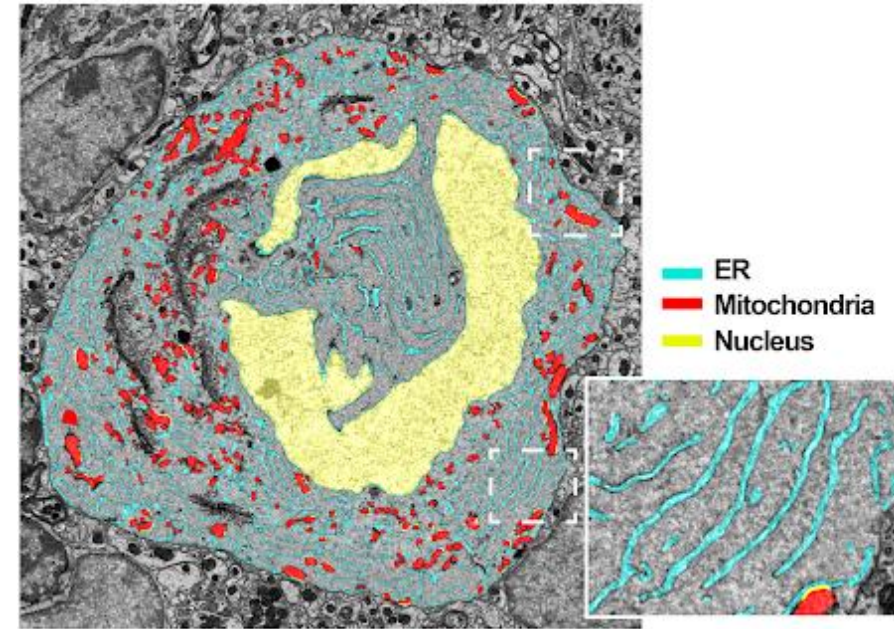
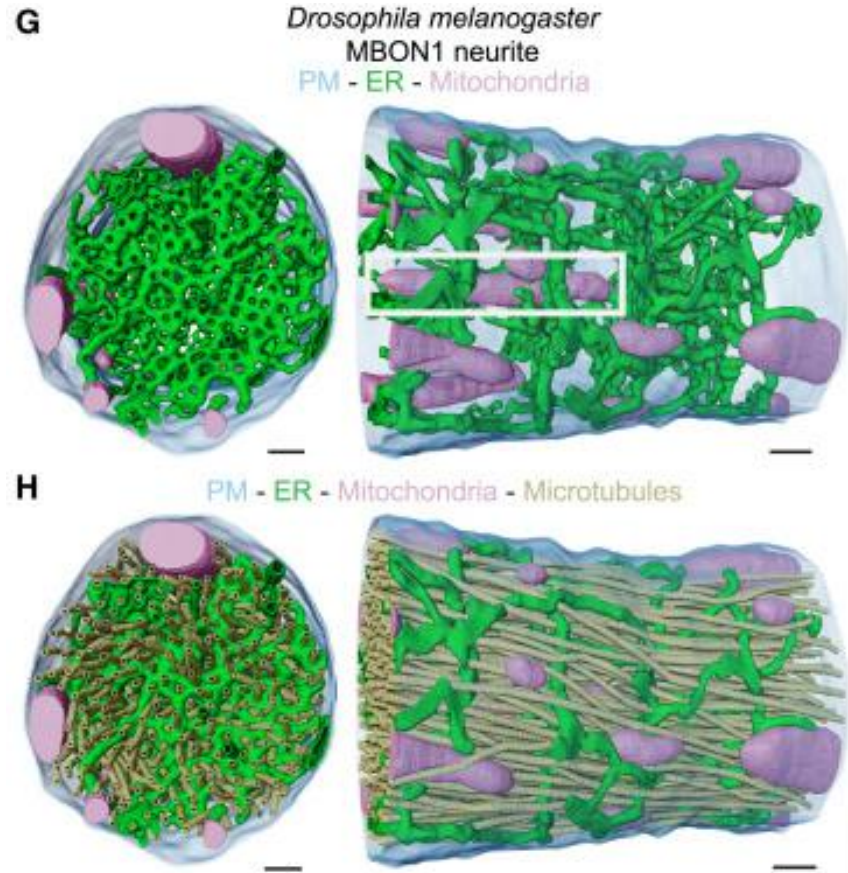


“Spot” the cow,
Keenan Crane 2013

(A cell is not a spherical cow)

Form and function in biological systems

- “Form follows function” (and/or vice versa)
- What are the implications of cell and organelle shape for cell signaling networks?

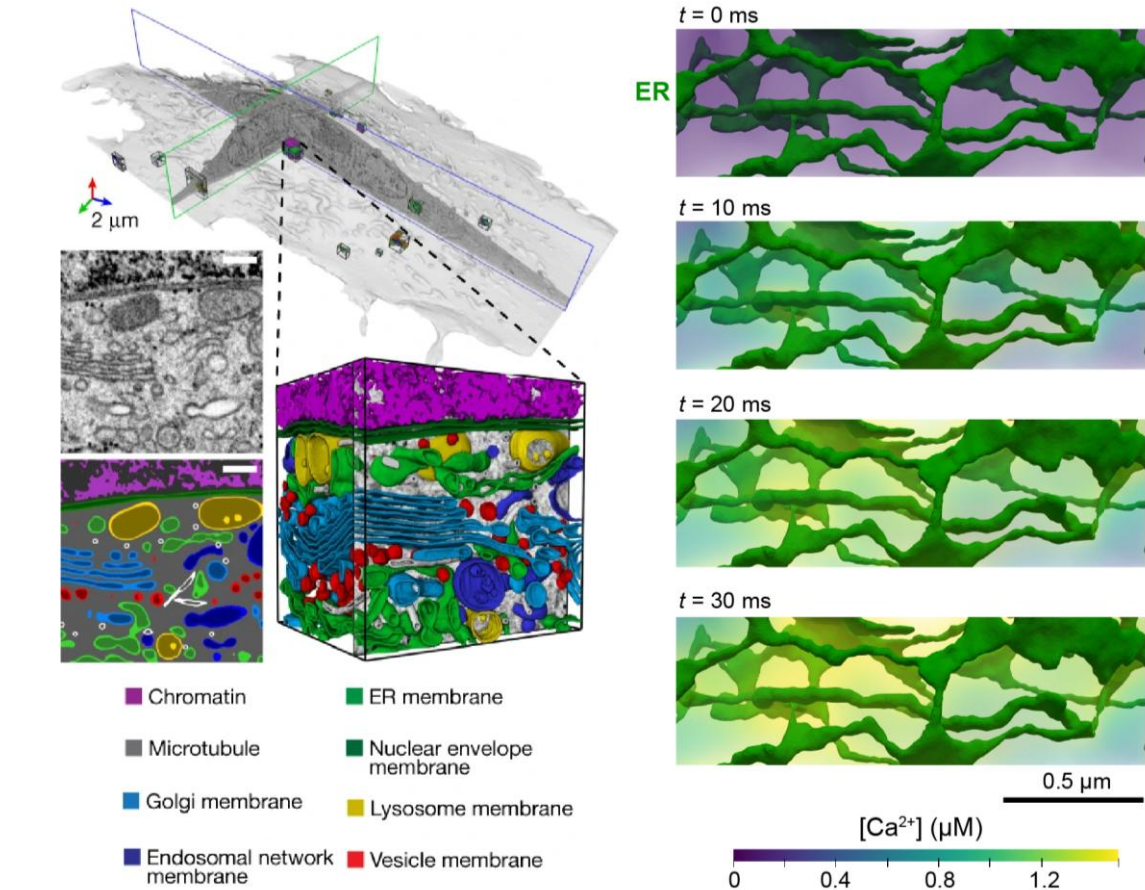
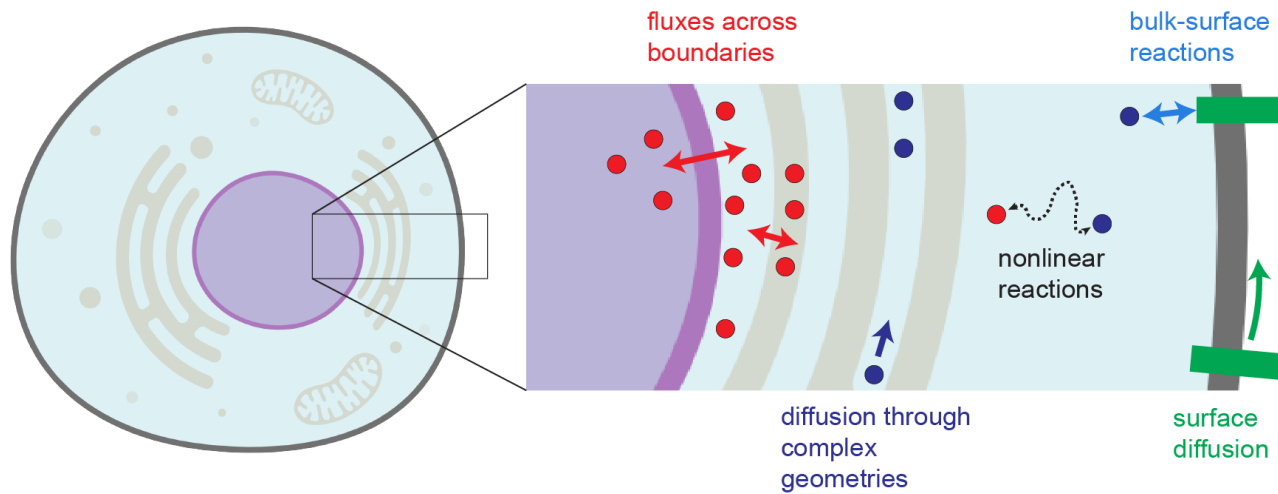


Haberl et al 2025, *Biorxiv*

Benedetti et al 2025, *Cell*

Reaction-transport systems in cell physiology

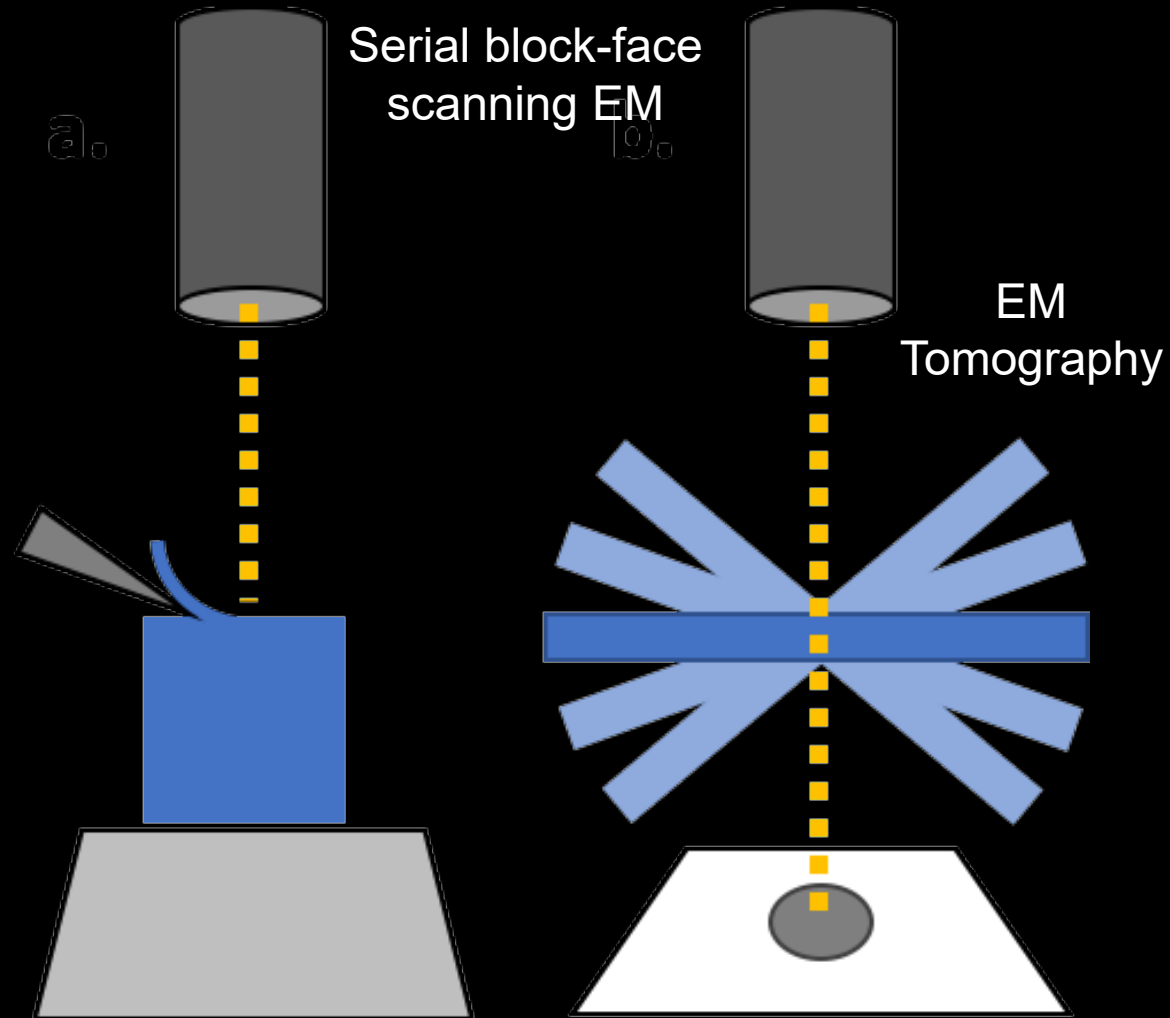
- Cell signaling networks are described by **mixed dimensional** PDEs governing reaction and transport (e.g., advection or diffusion) within cell geometries



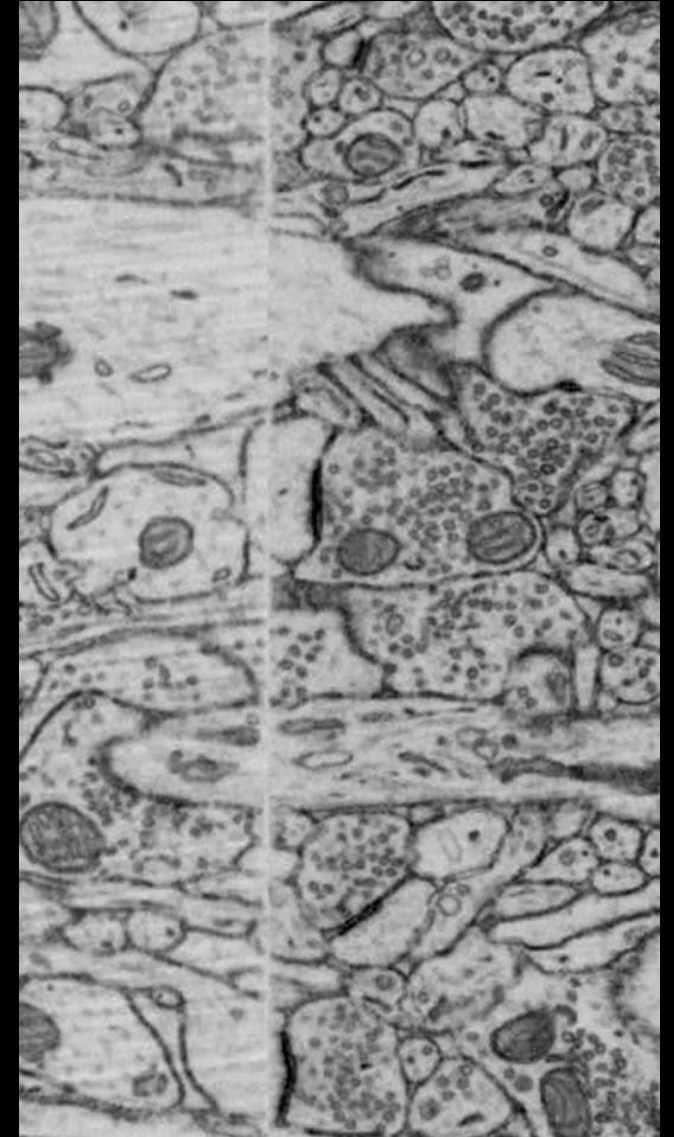
Outline of today's workshop

1. Overview of mesh conditioning for realistic cell geometries (GAMer2)
2. Overview of SMART (Spatial Modeling Algorithms for Reactions and Transport) mathematical framework and software structure
3. SMART software tutorial via JupyterHub
4. Break
5. Case study in SMART – diffusion of chemoattractant from a source particle

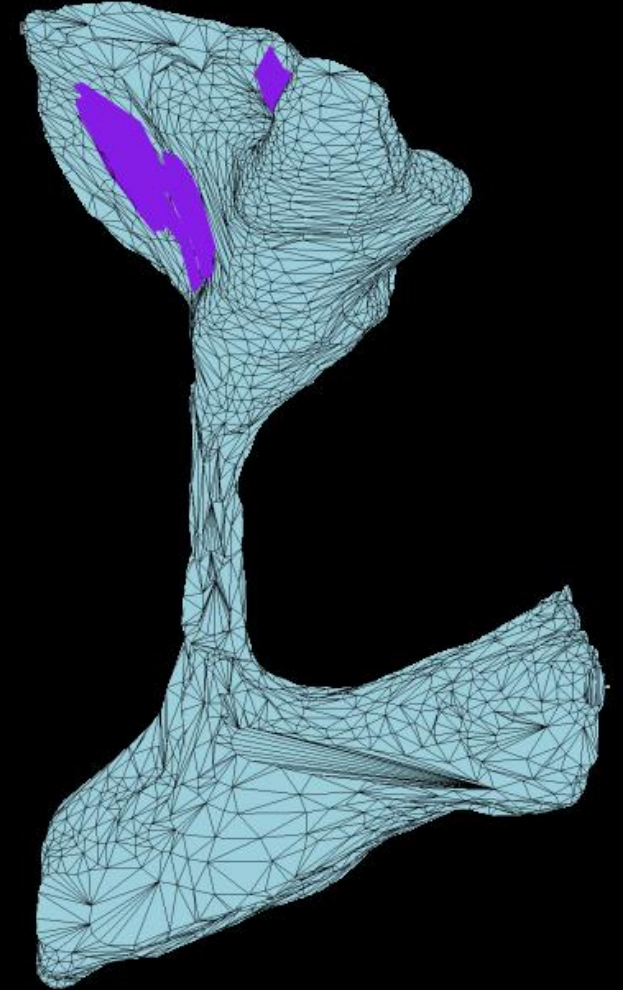
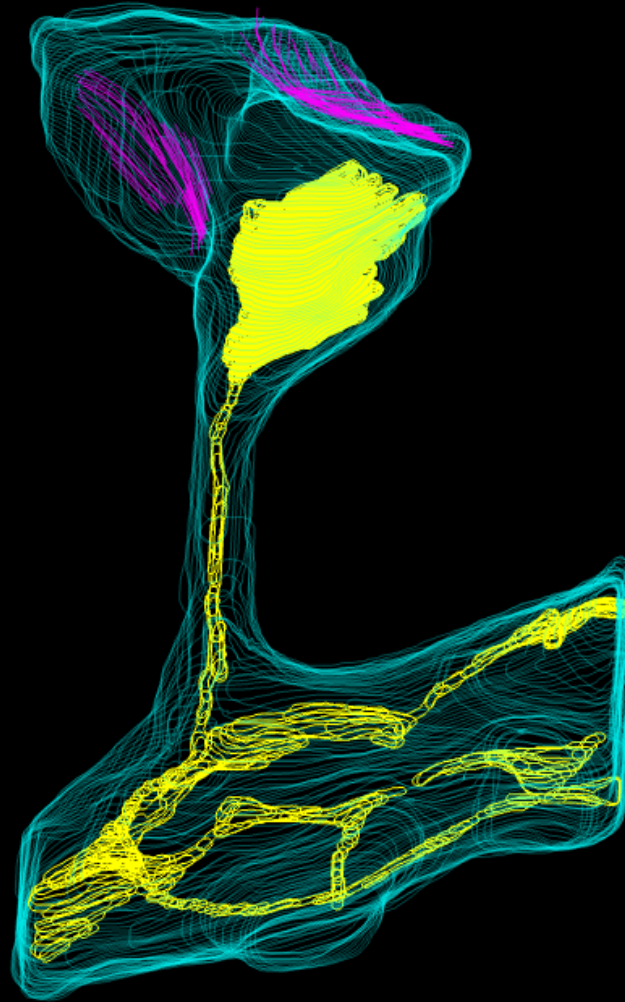
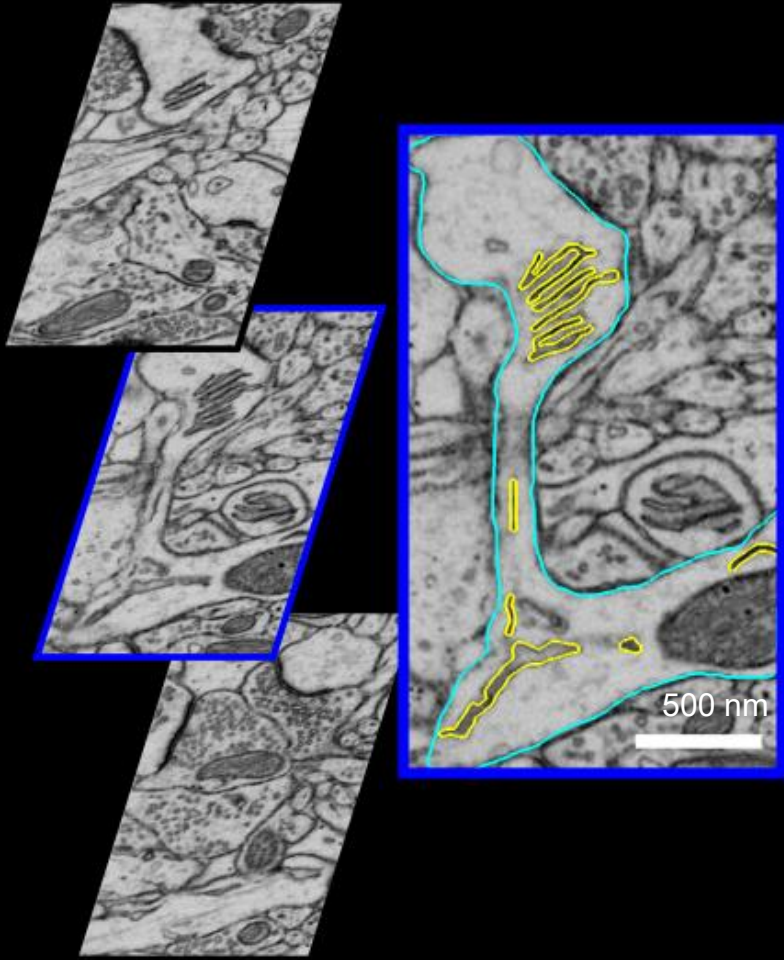
Volume EM is a powerful tool to interrogate cellular ultrastructure



Murine brain
FIB-SEM
Dataset



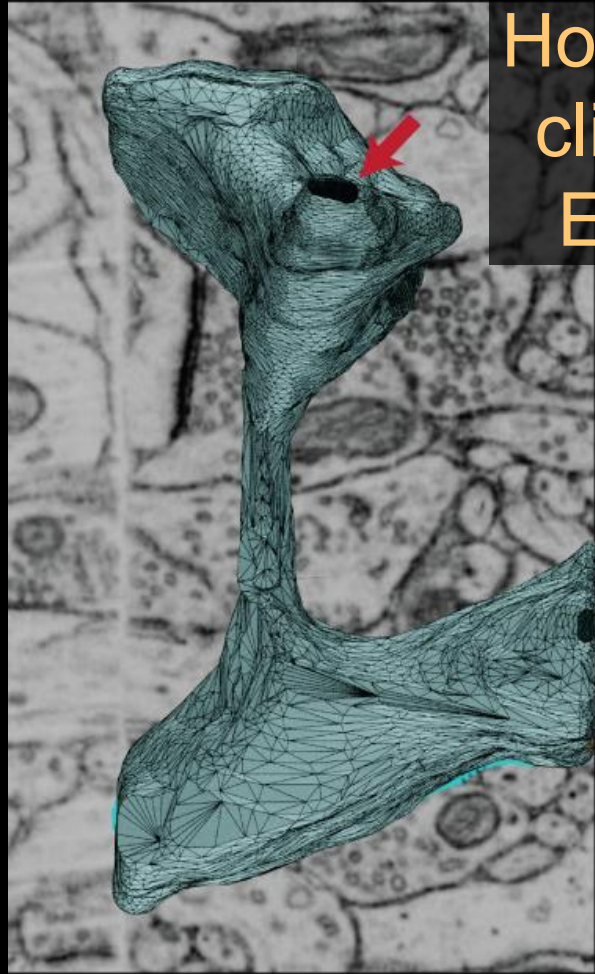
A mesh can be generated by segmenting features then contour tiling



IMOD2OBJ generated
meshes follow the
segmentation

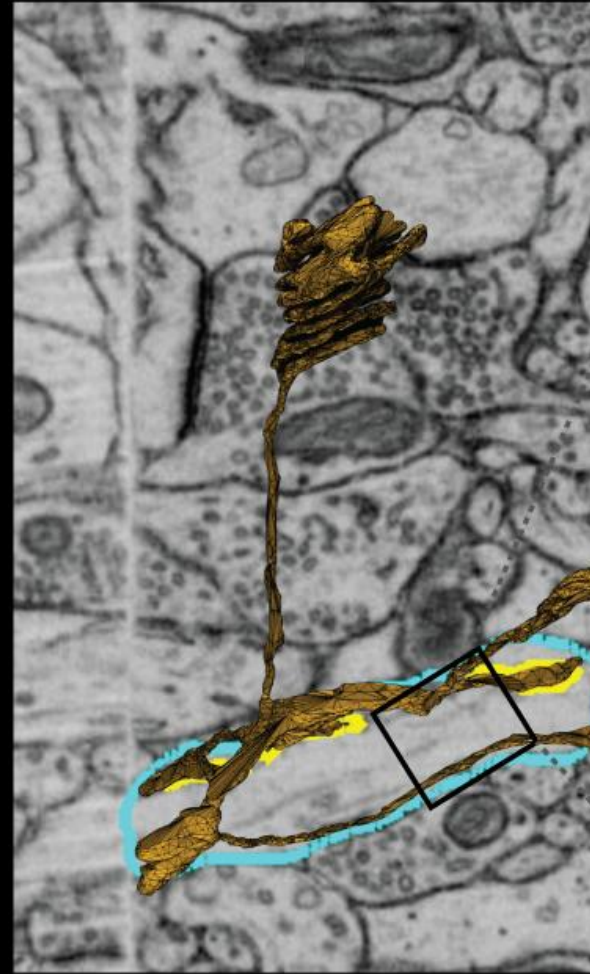


IMOD2OBJ meshes contain many artifacts



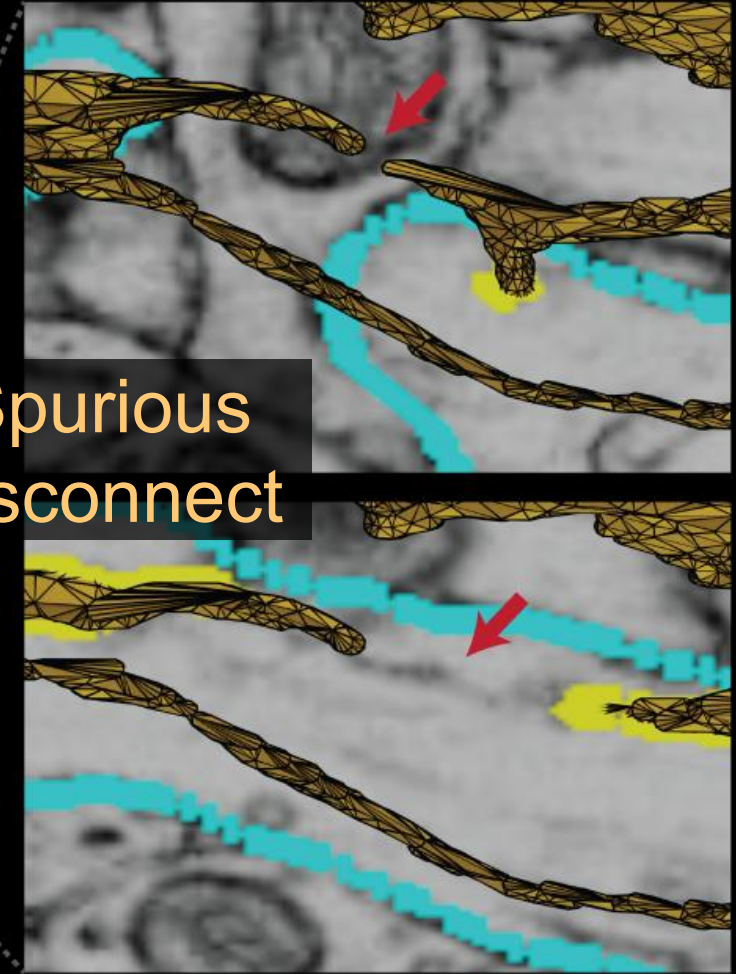
Holes due to
clipping by
EM stack

Plasma Membrane

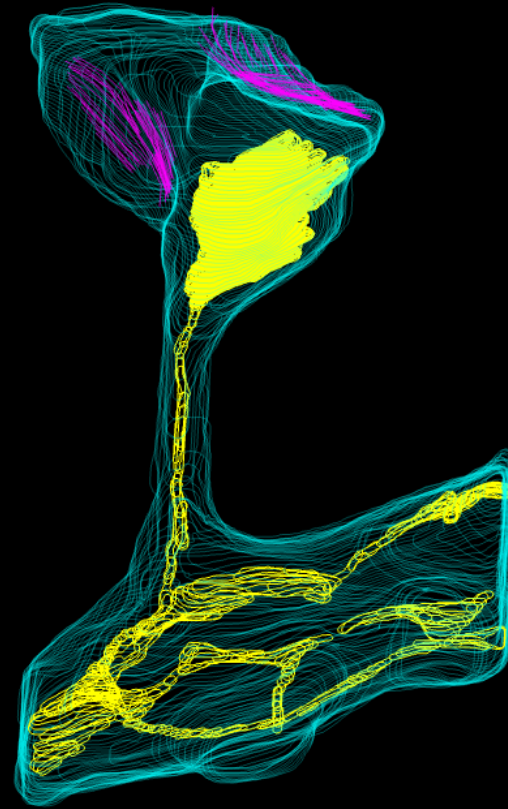
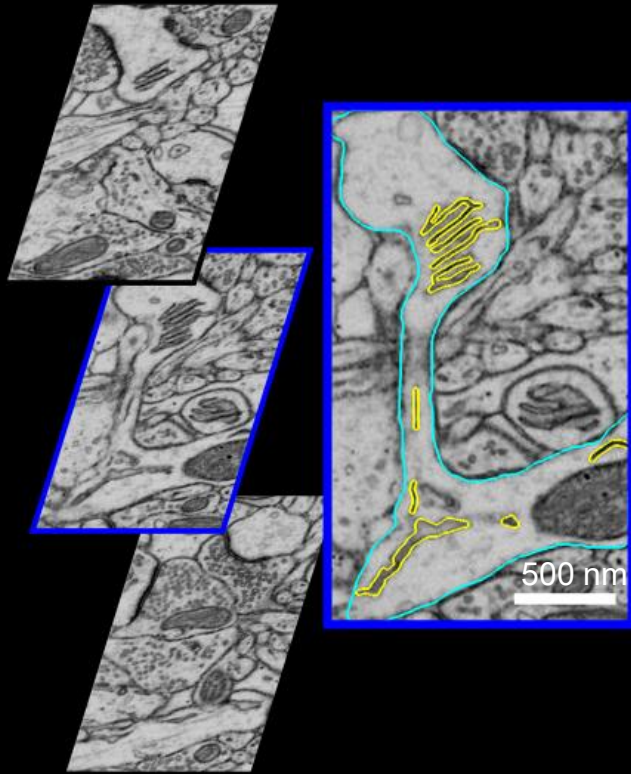


Spurious
disconnect

Endoplasmic Reticulum



Contour tiling algorithms generate poor quality mesh discretizations



Mesh conditioning↓ = stability↓ + accuracy↓ + CPU cost↑

Efficient solutions of PDEs using numerical methods are sensitive to the quality of discretization

Needed a solution satisfying:

- **Biologically relevant result**
- **Conditioned mesh**
- **Matches observed geometry**

Mesh conditioning↓ = stability↓ + accuracy↓ + CPU cost↑

GAMer 2: Geometry-Preserving Adaptive MeshER

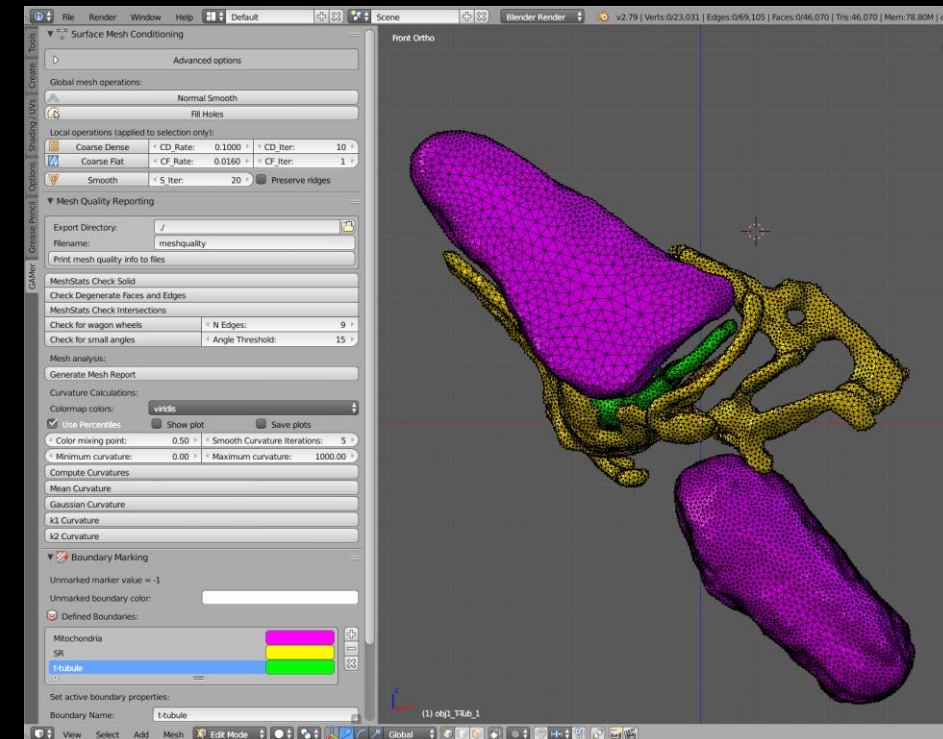
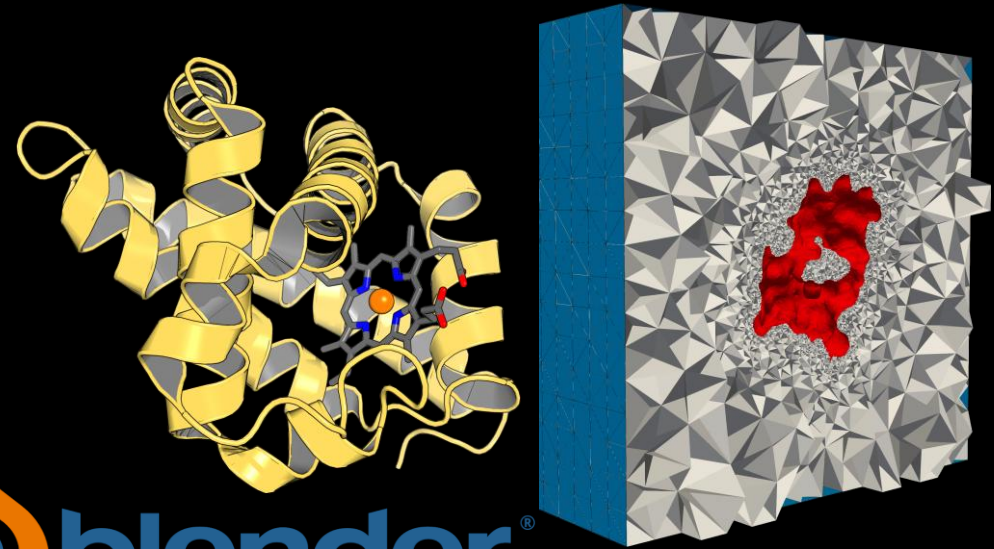
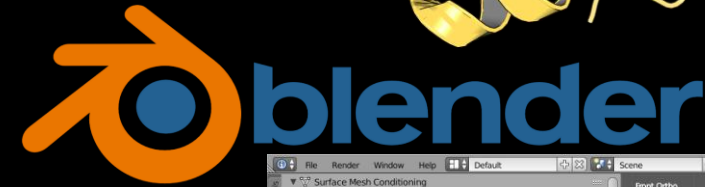


LGPL v 2.1 C++ Code Features

- Mesh conditioning
- Boundary marking
- Tetrahedralization (TetGen)

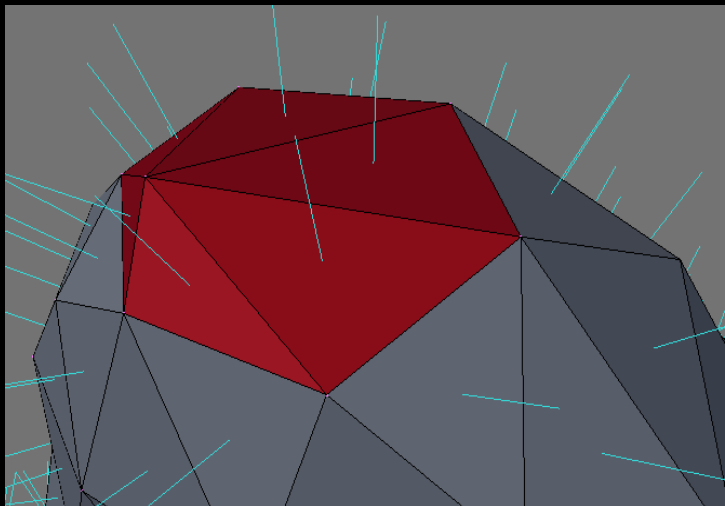
BlendGAMer Add-On

- Graphical feedback and advanced visualization
- Blender 3D mesh manipulation capability
- Mesh quality reporting
- Paint style boundary marking

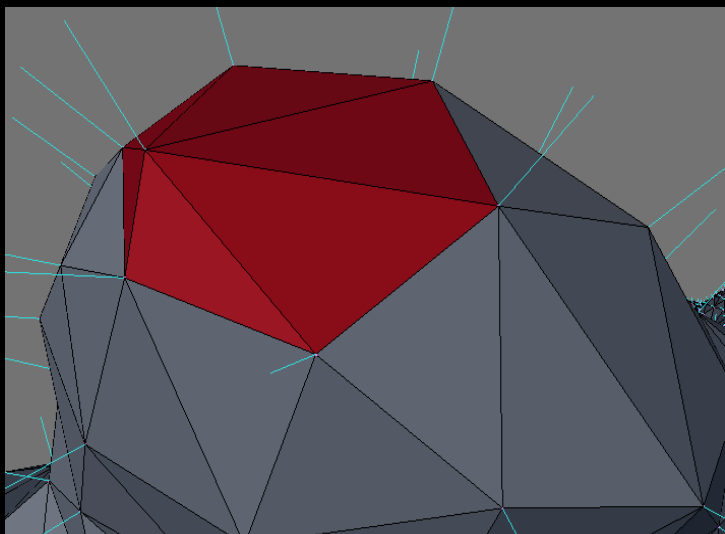


Local structure tensor is a rough measure of curvature

Triangle Normals, $\mathbf{n}_t$



Vertex Normals, $\mathbf{n}$



Vertex normal is the average of incident triangle normal:

$$\mathbf{n} = \frac{1}{N} \sum_{i=0}^{N_t} \mathbf{n}_t$$

Local structure tensor:

$$T(\mathbf{v}) = \sum_{i=1}^{N_r} \mathbf{n}_i \otimes \mathbf{n}_i = \sum_{i=1}^{N_r} \begin{pmatrix} n_i^x n_i^x & n_i^x n_i^y & n_i^x n_i^z \\ n_i^y n_i^x & n_i^y n_i^y & n_i^y n_i^z \\ n_i^z n_i^x & n_i^z n_i^y & n_i^z n_i^z \end{pmatrix}$$

With eigenvalues:

$$\lambda_0 \geq \lambda_1 \geq \lambda_2$$

Corresponding Cases:

Planes: $\lambda_0 \gg \lambda_1 \approx \lambda_2 \approx 0$

Ridges and valleys: $\lambda_0 \approx \lambda_1 \gg \lambda_2 \approx 0$

Spheres and saddles: $\lambda_0 \approx \lambda_1 \approx \lambda_2 > 0$

Vertex smoothing algorithm is a modified Laplacian smooth which optimizes angles

1. Define weight factor which prioritizes contribution from small angles:

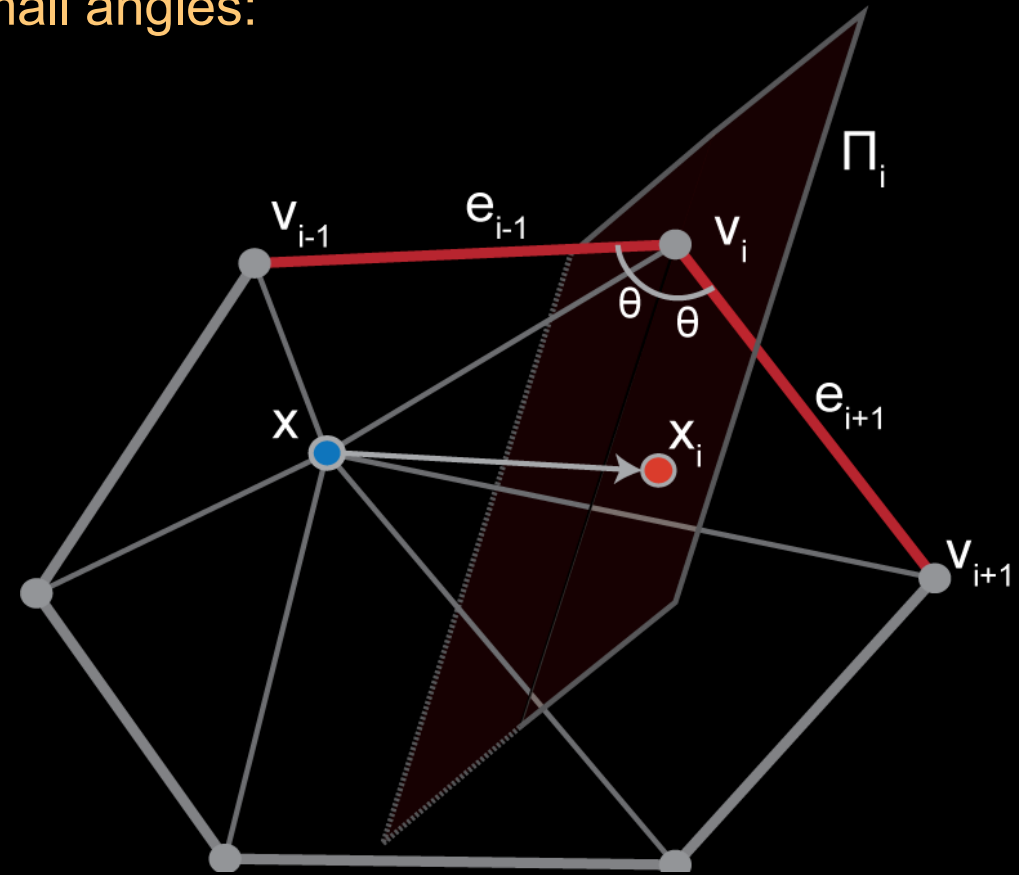
$$\frac{1}{\angle(\mathbf{v}_{i-1}, \mathbf{v}_i, \mathbf{v}_{i+1})} \propto \alpha_i \equiv \frac{\mathbf{e}_{i-1} \cdot \mathbf{e}_{i+1}}{|\mathbf{e}_{i-1}| \cdot |\mathbf{e}_{i+1}|}$$

2. Compute new weighted vertex position:

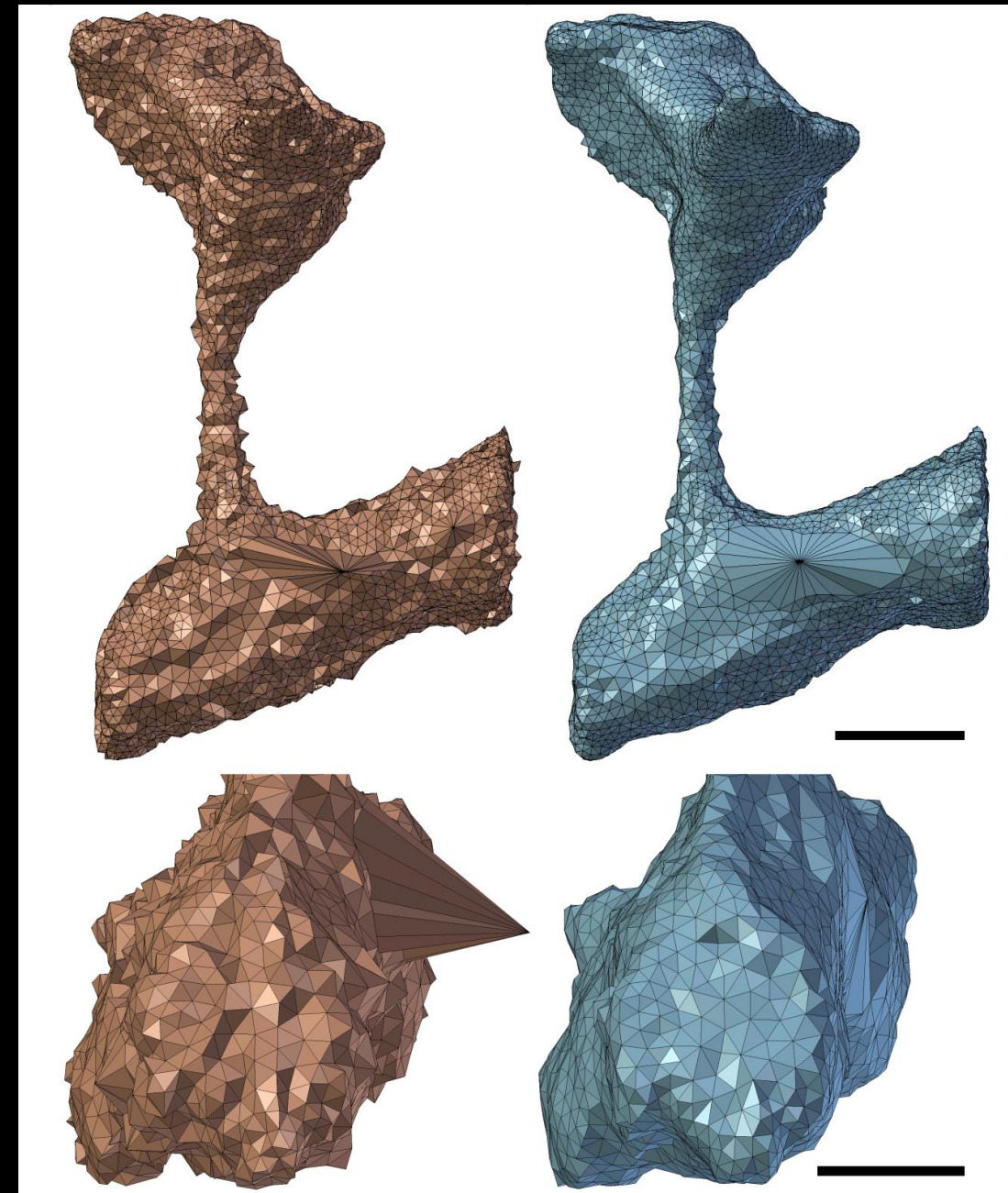
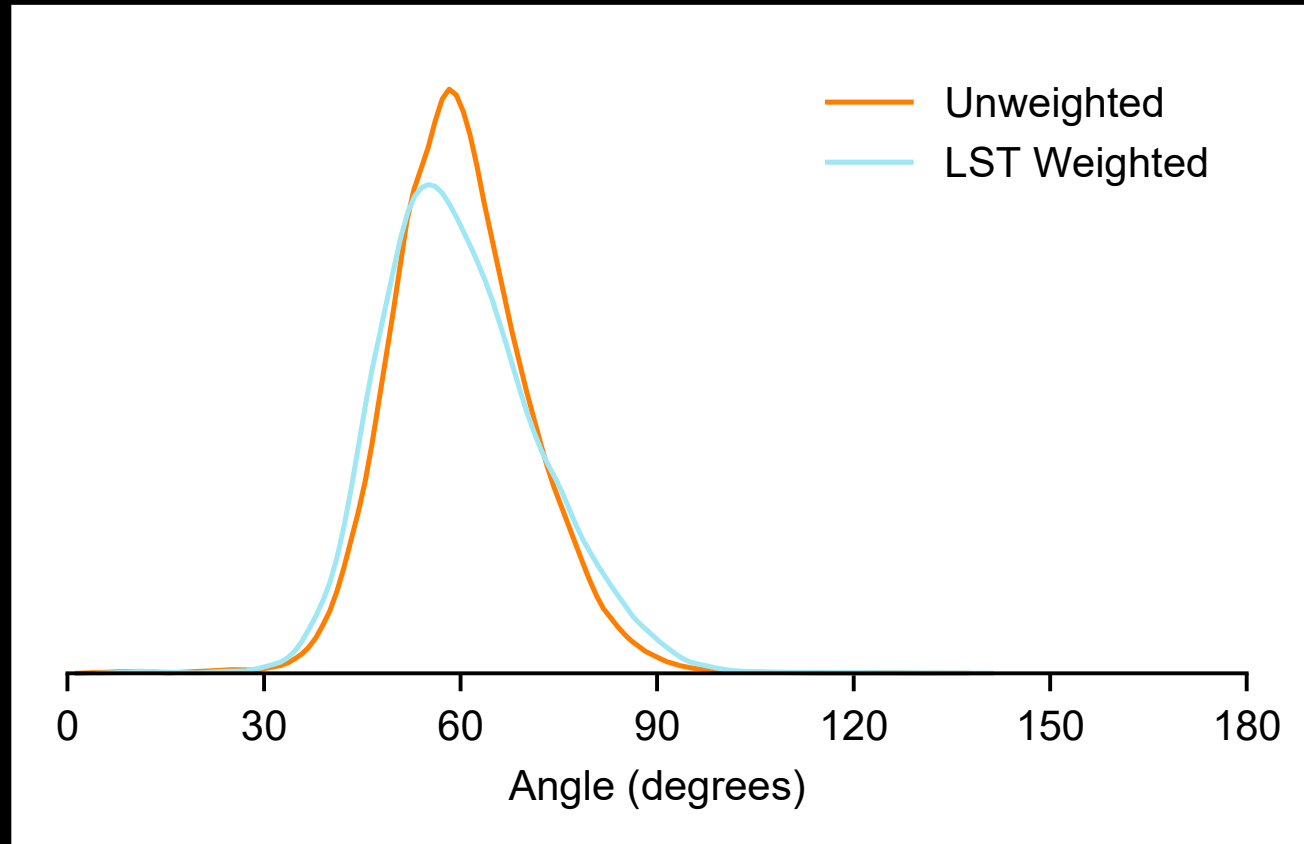
$$\bar{\mathbf{x}} = \frac{1}{\sum_{i=1}^N (\alpha_i + 1)} \sum_{i=1}^N (\alpha_i + 1) \mathbf{x}_i$$

3. Restrict diffusion by principal directions of LST:

$$\hat{\mathbf{x}} = \mathbf{x} + \sum_{k=1}^3 \frac{1}{1 + \lambda_k} [(\bar{\mathbf{x}} - \mathbf{x}) \cdot \mathbf{E}_k] \mathbf{E}_k$$



Local structure tensor projection helps preserve geometry



Unweighted

LST Weighted

Anisotropic normal smooth produces smoothly varying vertex normals

1. Compute normal of incident face:

$$\bar{\mathbf{n}}_i = \sum_{j=1}^3 \mathbf{n}_{ij} / 3$$

2. Compute rotation of $\mathbf{x}$ around $\mathbf{e}_i$ to align normals:

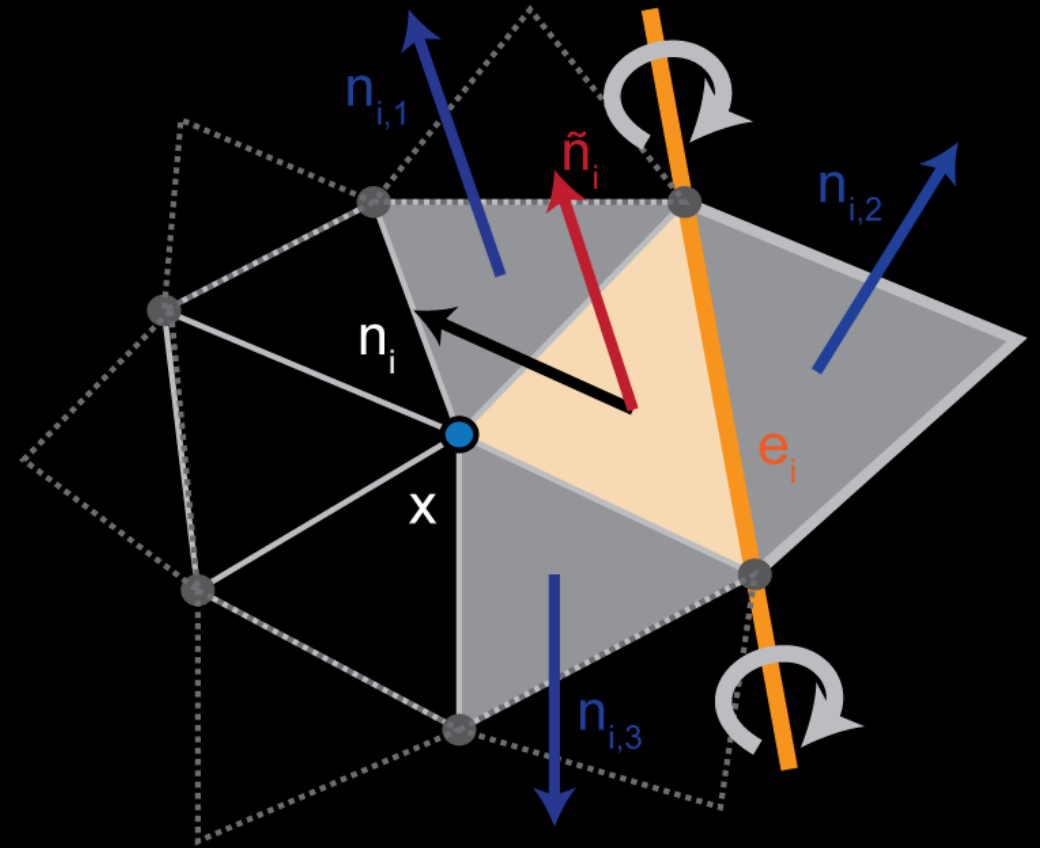
$$R(\mathbf{x}; \mathbf{e}_i, \theta_i)$$

3. Sum contributions across incident faces:

$$\bar{\mathbf{x}} = \frac{1}{\sum_{i=1}^{N_1} a_i} \sum_{i=1}^{N_1} a_i R(\mathbf{x}; \mathbf{e}_i, \theta_i)$$

4. Anisotropically weight to preserve sharp features:

$$\bar{\mathbf{n}}_i = \frac{1}{\sum_{j=1}^3 e^{K(\mathbf{n}_i \cdot \mathbf{n}_{ij})}} \sum_{j=1}^3 e^{K(\mathbf{n}_i \cdot \mathbf{n}_{ij})} \mathbf{n}_{ij}$$



Mesh decimation removes vertices and retriangulates the hole

1. Select vertices to remove:

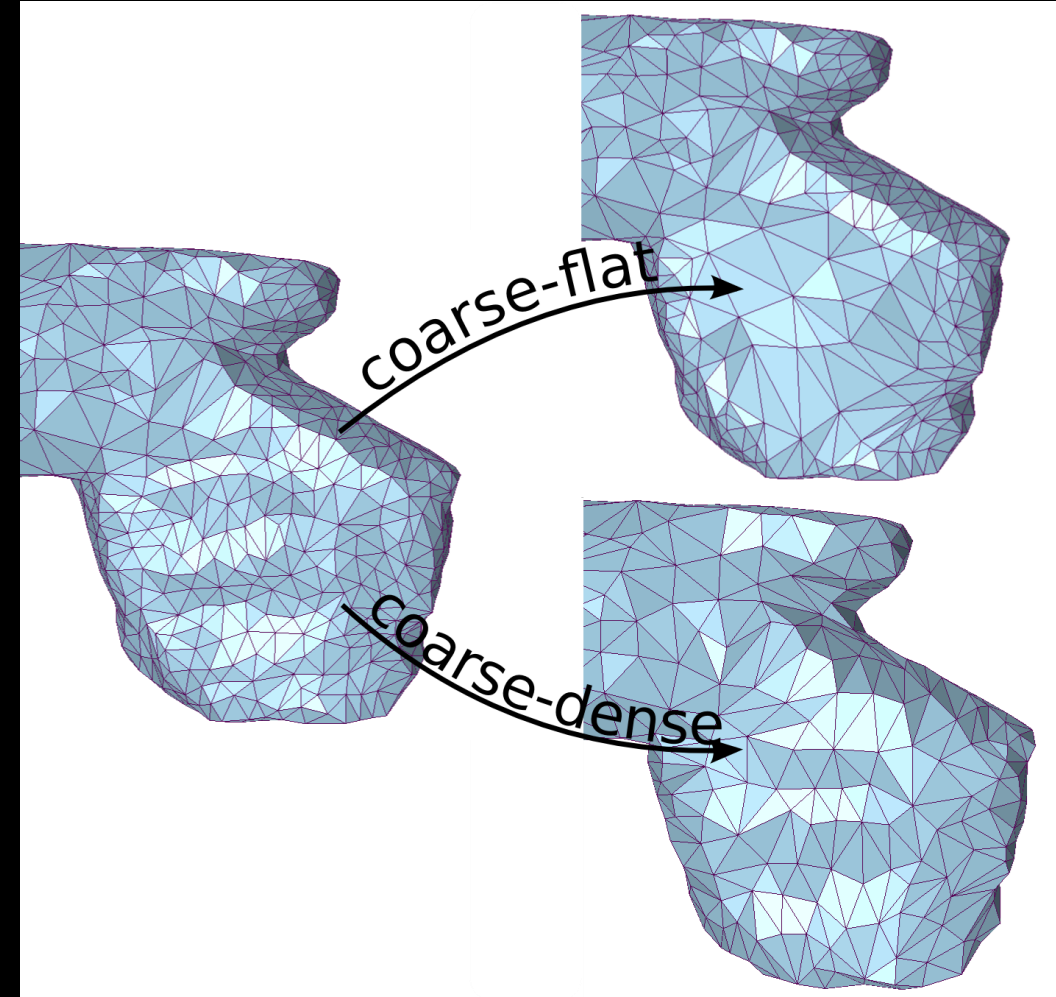
Criteria by flatness:

$$\frac{\lambda_2}{\lambda_1} < R_1 \quad \text{Planes: } \lambda_0 \gg \lambda_1 \approx \lambda_2 \approx 0$$

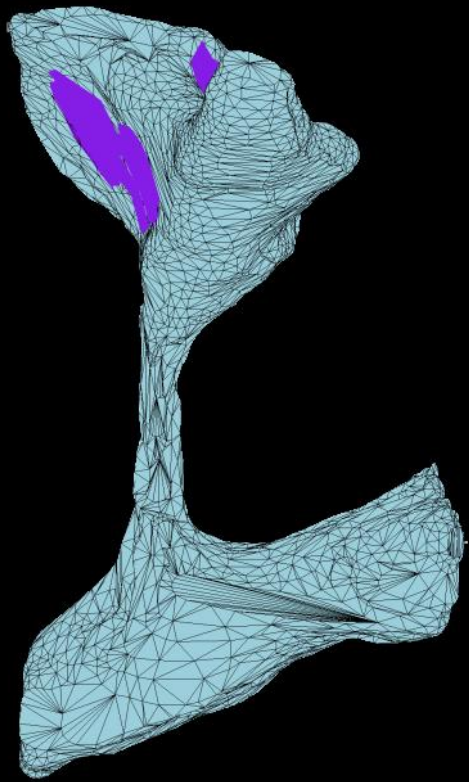
Criteria by denseness (edge length):

$$\frac{\max_{i=1}^{N_1} d(\vec{x}, \vec{v}_i)}{\overline{D}} < R_2$$

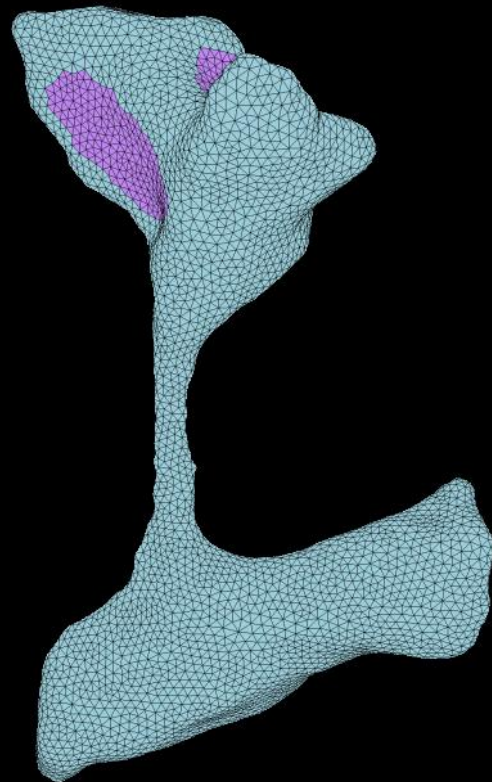
2. Recursive triangulation of hole



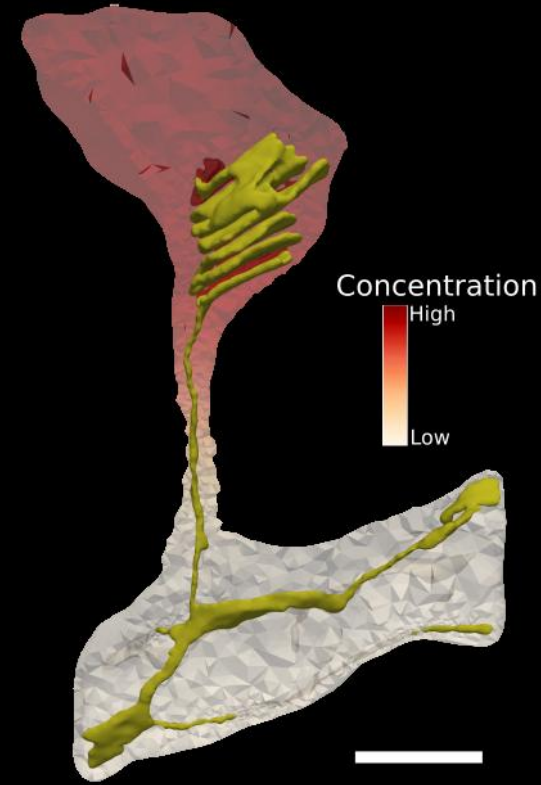
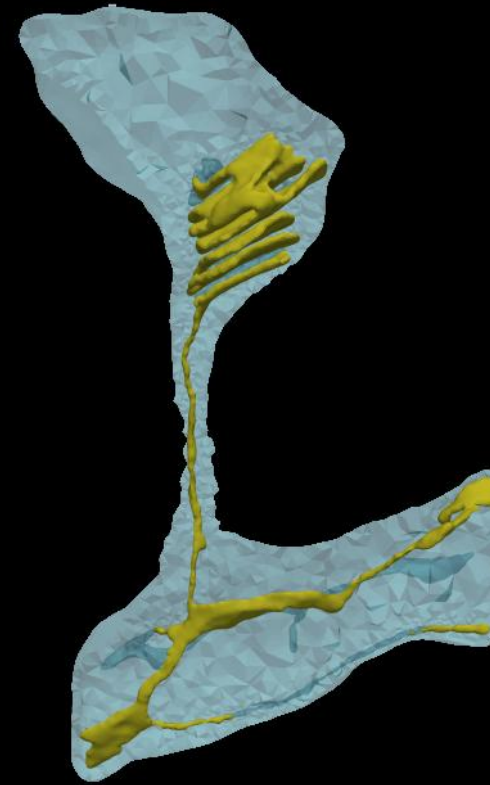
GAMer 2 enables physical simulations with realistic geometries



1) Mesh Conditioning
2) Boundary Marking



3) Tetrahedralization
(TetGen)

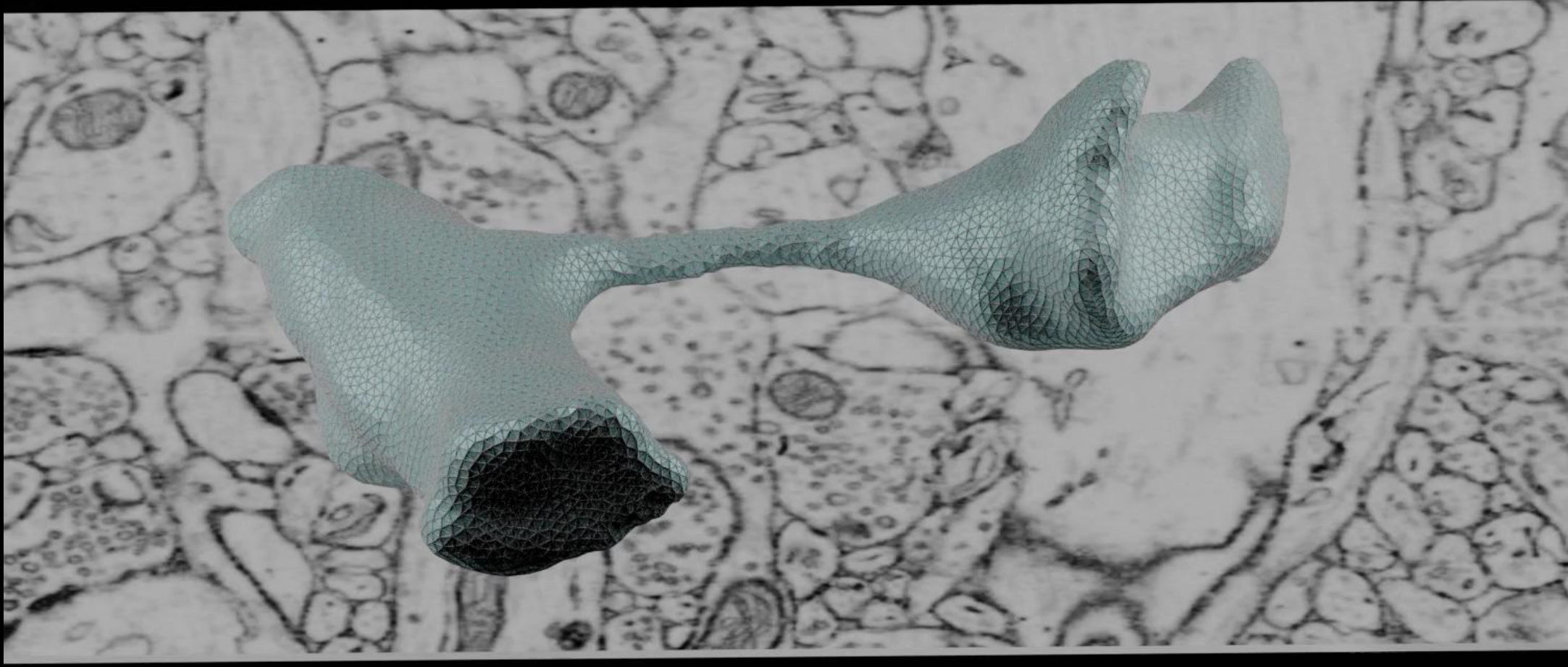


Concentration
High
Low

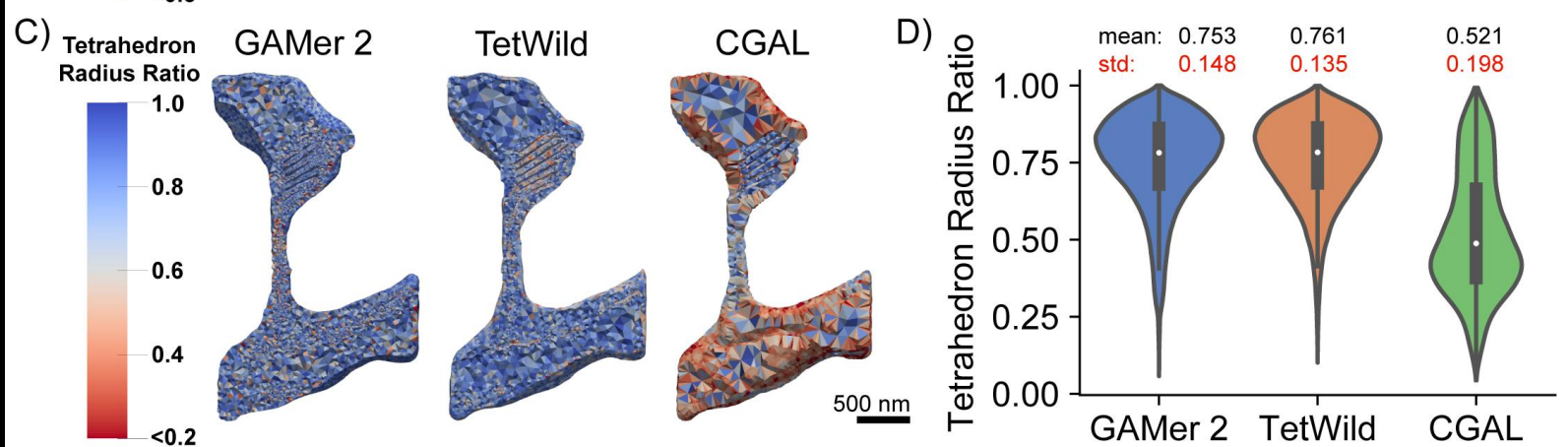
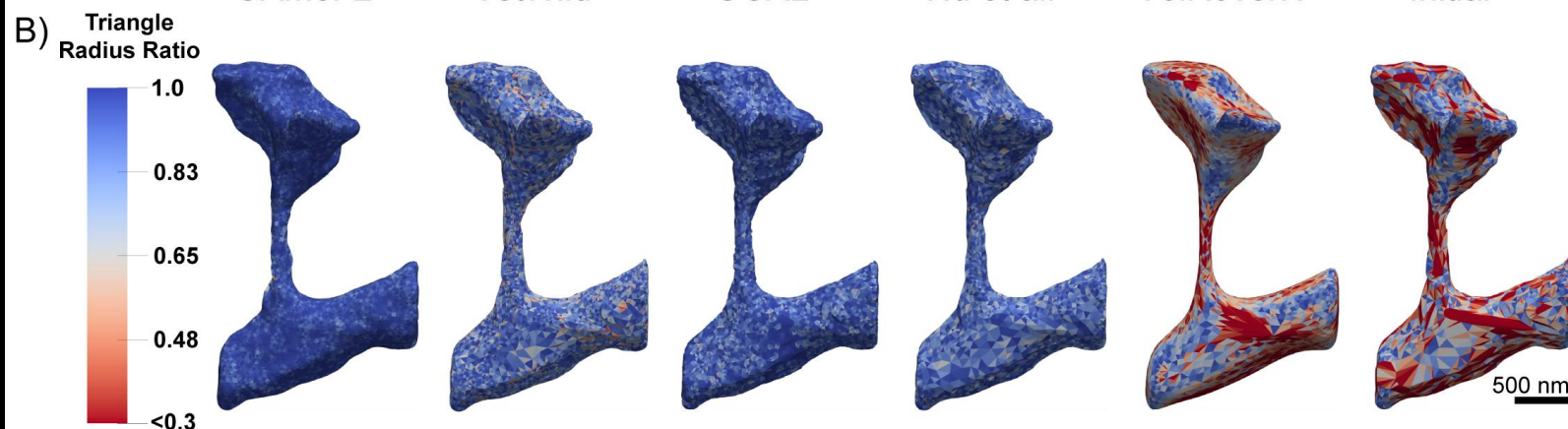
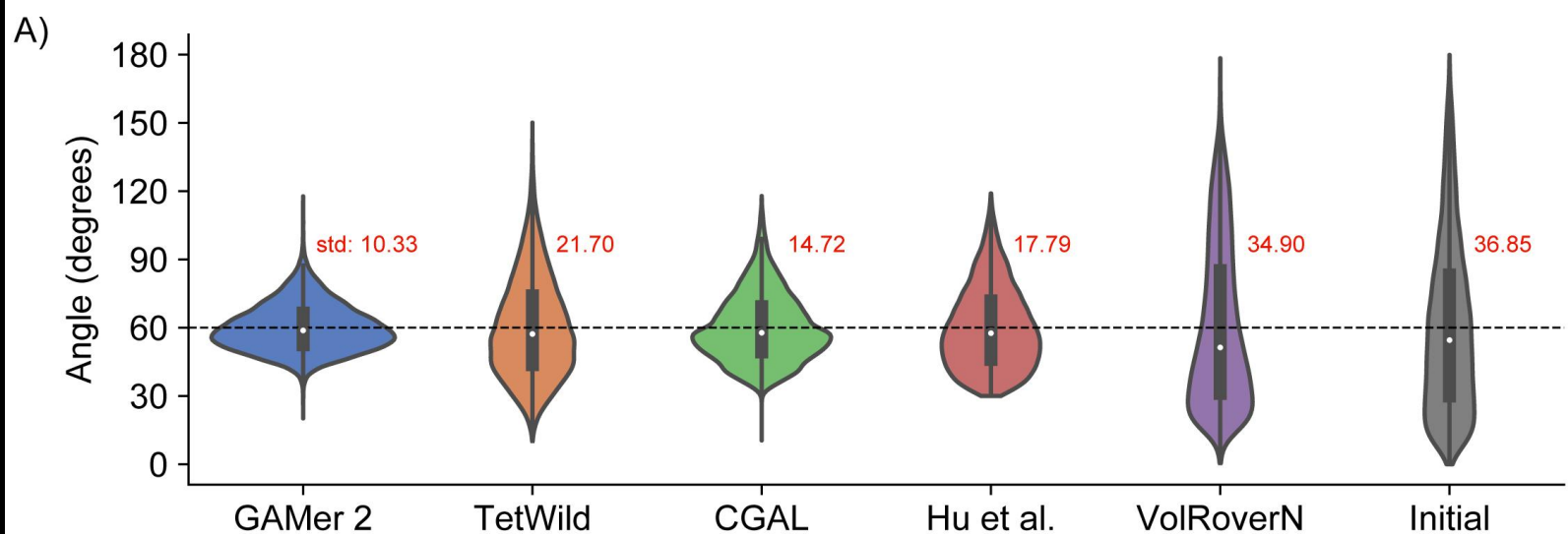


FENICS
PROJECT

GAMer meshes preserve geometric features

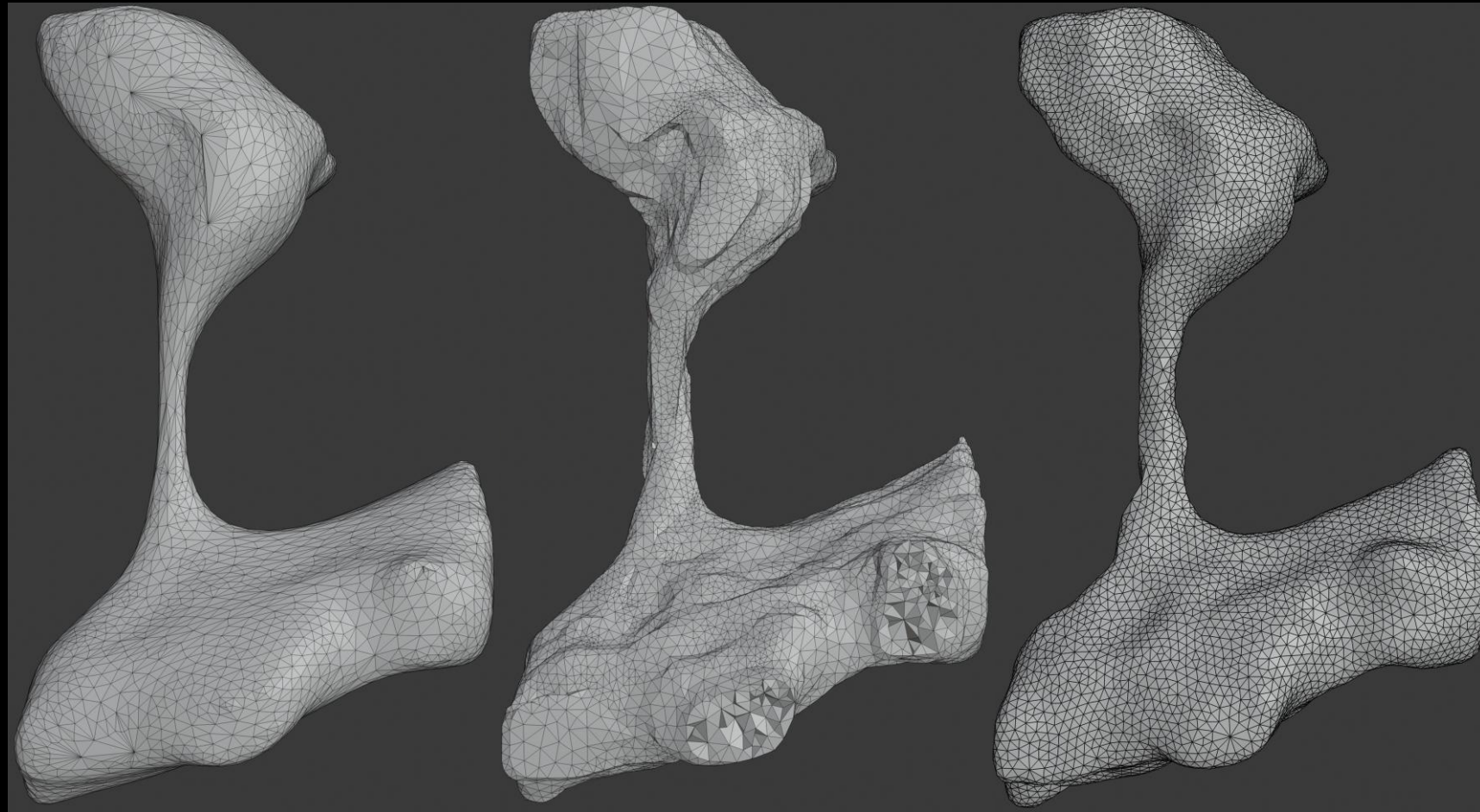


GAMer produced meshes are competitively high quality



GAMer: CTL*; JGL*; et al.; PLOS Comp. Biol. 2020
 CGAL Mesh_3: Alliez, P.; et al; CGAL Software 5.0 (cgal.org)
 VolRoverN: Edwards, J.; et al.; Neuroinform 2014
 TetWild: Hu, Y.; et al.; ACM Trans. Graph. 2018
 TVCG: Hu, K.; et al.; IEEE Trans. Visual. Comput. Graphics 2017

Meshes
generated by
other tools may
have good
conditioning but
poor biological
significance



VolRoverN

TetWild

GAMer 2

GAMer: CTL*; JGL*; et al.; PLOS Comp. Biol. 2020
CGAL Mesh_3: Alliez, P; et al; CGAL Software 5.0 (cgal.org)
VolRoverN: Edwards, J; et al.; Neuroinform 2014
TetWild: Hu, Y.; et al.; ACM Trans. Graph. 2018
TVCG: Hu, K.; et al.; IEEE Trans. Visual. Comput. Graphics 2017

Formulation of mixed dimensional reaction-transport systems

Consider a volumetric compartment, Ω^m , with boundary Γ^q and normal $\mathbf{n}^m$, and adjacent to other volumetric compartments Ω^n

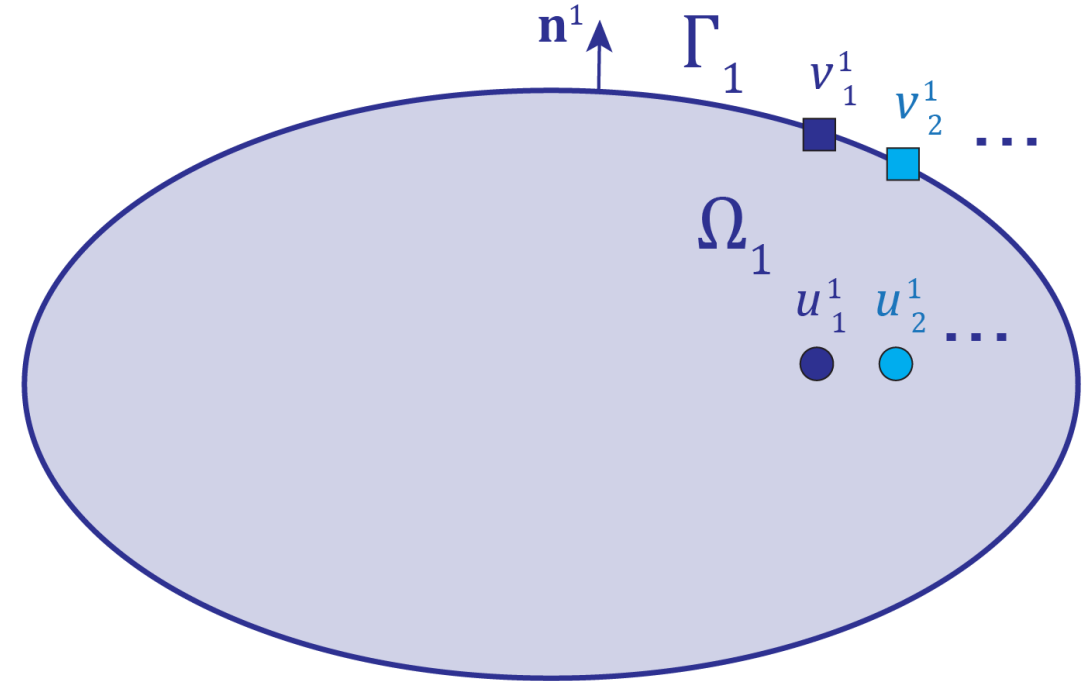
For each species in this compartment, with concentration u_i^m ,

Diffusion	Volume reactions	
$\partial_t u_i^m - \nabla \cdot (D_i \nabla u_i^m) - f_i^m(u^m) = 0$		in Ω^m

Diffusive flux	Surface reactions	
$D_i \nabla u_i^m \cdot \mathbf{n}^m - R_i^q(u^m, u^n, v^q) = 0$		on Γ^q

For each species on the boundary of this compartment, with concentration v_j^q ,

Surface diffusion	Surface reactions	
$\partial_t v_j^q - \nabla_\Gamma \cdot (D_j \nabla_\Gamma v_j^q) - g_j^q(u^m, u^n, v^q) = 0$		on Γ^q



Formulation of mixed dimensional reaction-transport systems

Consider a volumetric compartment, Ω^m , with boundary Γ^q and normal $\mathbf{n}^m$, and adjacent to other volumetric compartments Ω^n

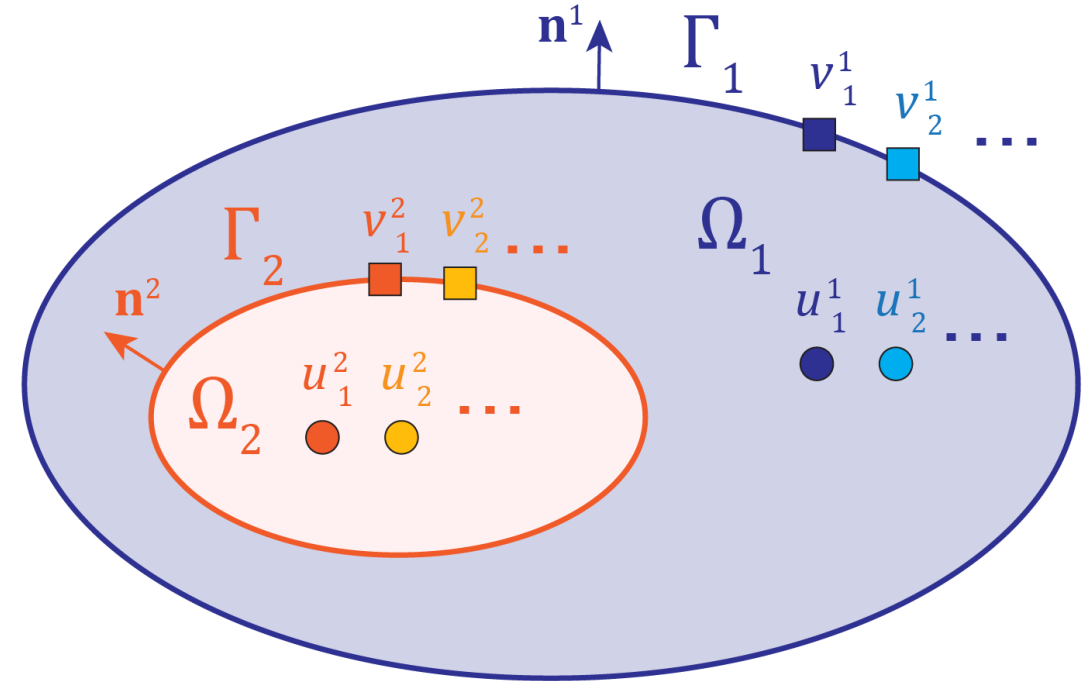
For each species in this compartment, with concentration u_i^m ,

Diffusion	Volume reactions	
$\partial_t u_i^m - \nabla \cdot (D_i \nabla u_i^m) - f_i^m(u^m) = 0$		in Ω^m

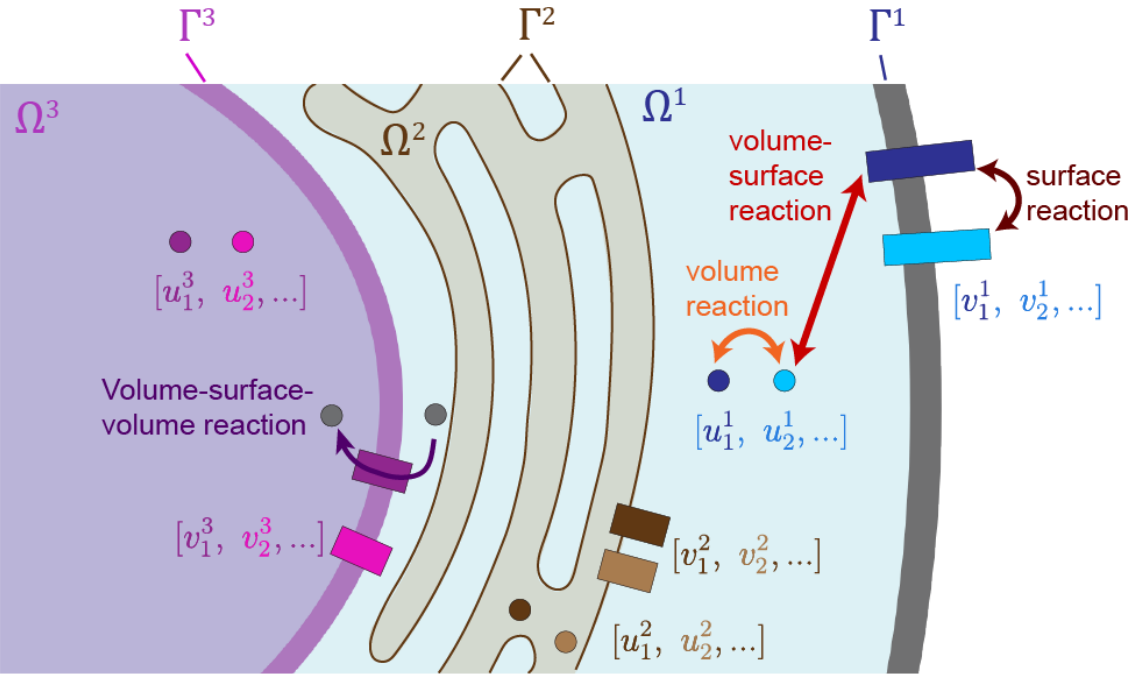
Diffusive flux	Surface reactions	
$D_i \nabla u_i^m \cdot \mathbf{n}^m - R_i^q(u^m, u^n, v^q) = 0$		on Γ^q

For each species on the boundary of this compartment, with concentration v_j^q ,

Surface diffusion	Surface reactions	
$\partial_t v_j^q - \nabla_\Gamma \cdot (D_j \nabla_\Gamma v_j^q) - g_j^q(u^m, u^n, v^q) = 0$		on Γ^q



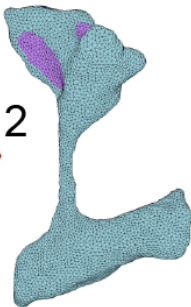
Spatial Modeling Algorithms for Reaction and Transport (SMART)



Reaction type	Example
Volume	Calcium binding to cytosolic buffers
Surface	Change in ATP synthase conformation
Volume-surface	Calcium binding to membrane-fixed buffers
Volume-surface-volume	YAP/TAZ nuclear translocation



Gmsh



GAMer 2



Load or generate mesh (Gmsh / GAMer 2)

Define species:

- Initial conditions
- Diffusion coefficients
- Compartments

Define reactions:

- Reactants
- Products
- Reaction type - mass action or custom

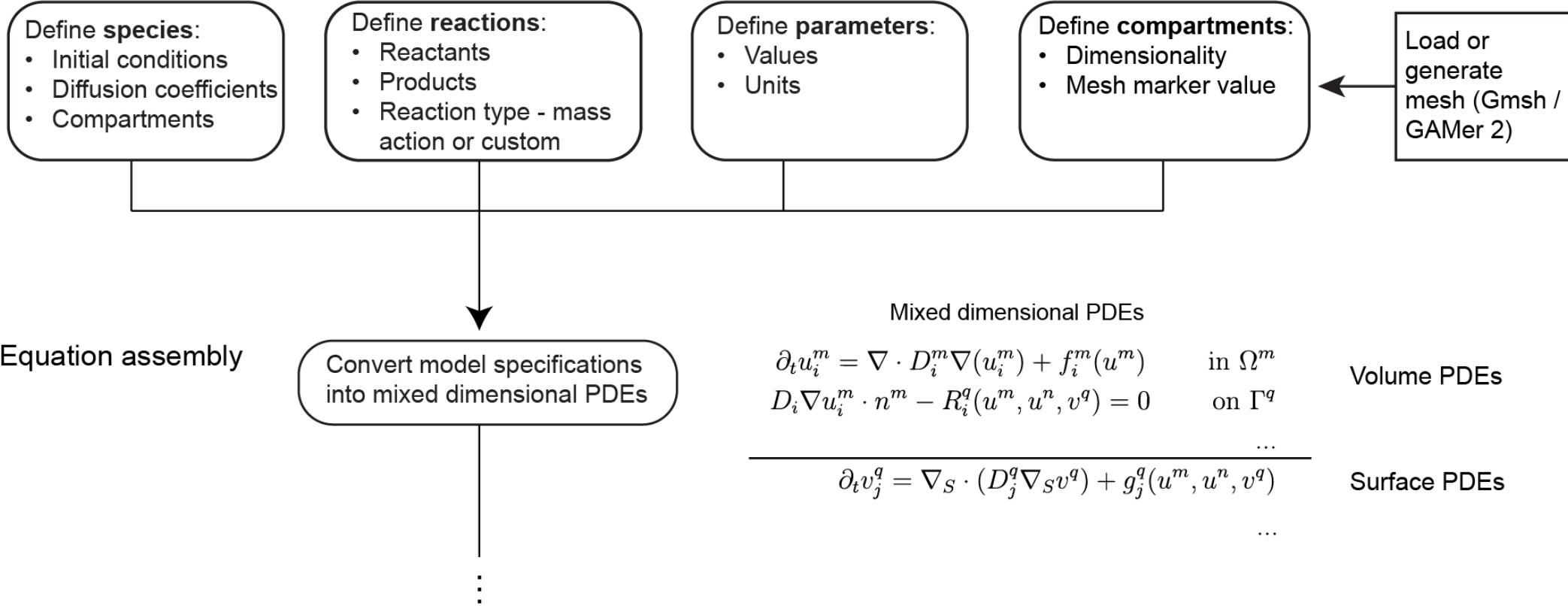
Define parameters:

- Values
- Units

Define compartments:

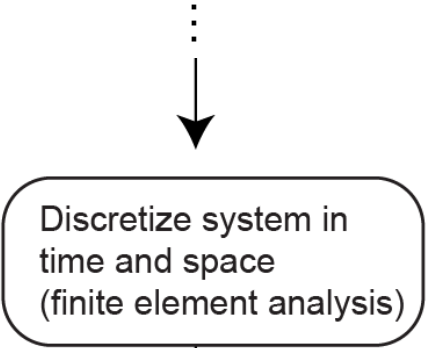
- Dimensionality
- Mesh marker value

Spatial Modeling Algorithms for Reaction and Transport (SMART)



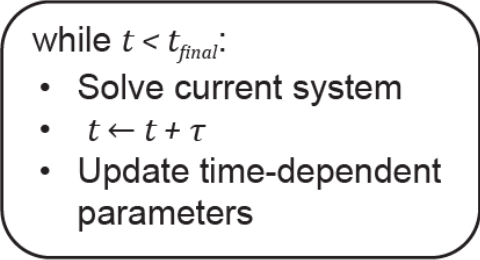
Spatial Modeling Algorithms for Reaction and Transport (SMART)

Model discretization

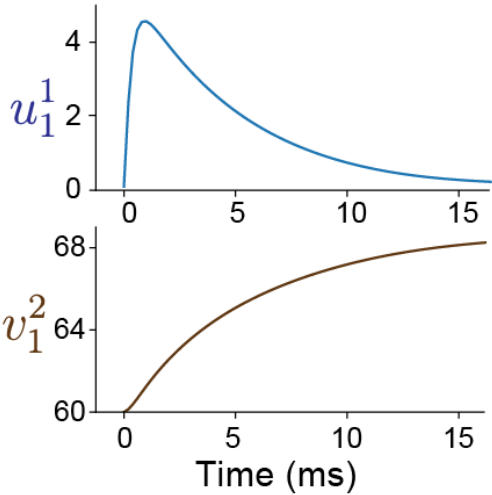
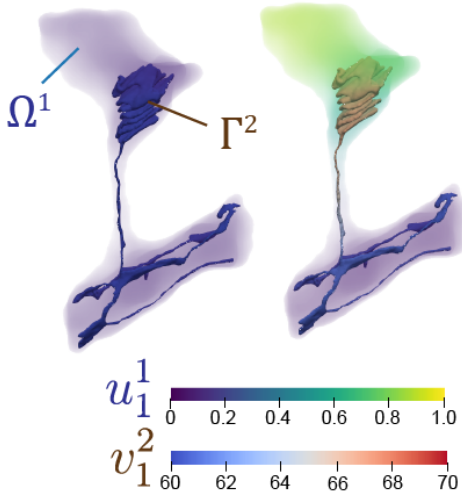


$t = 0$

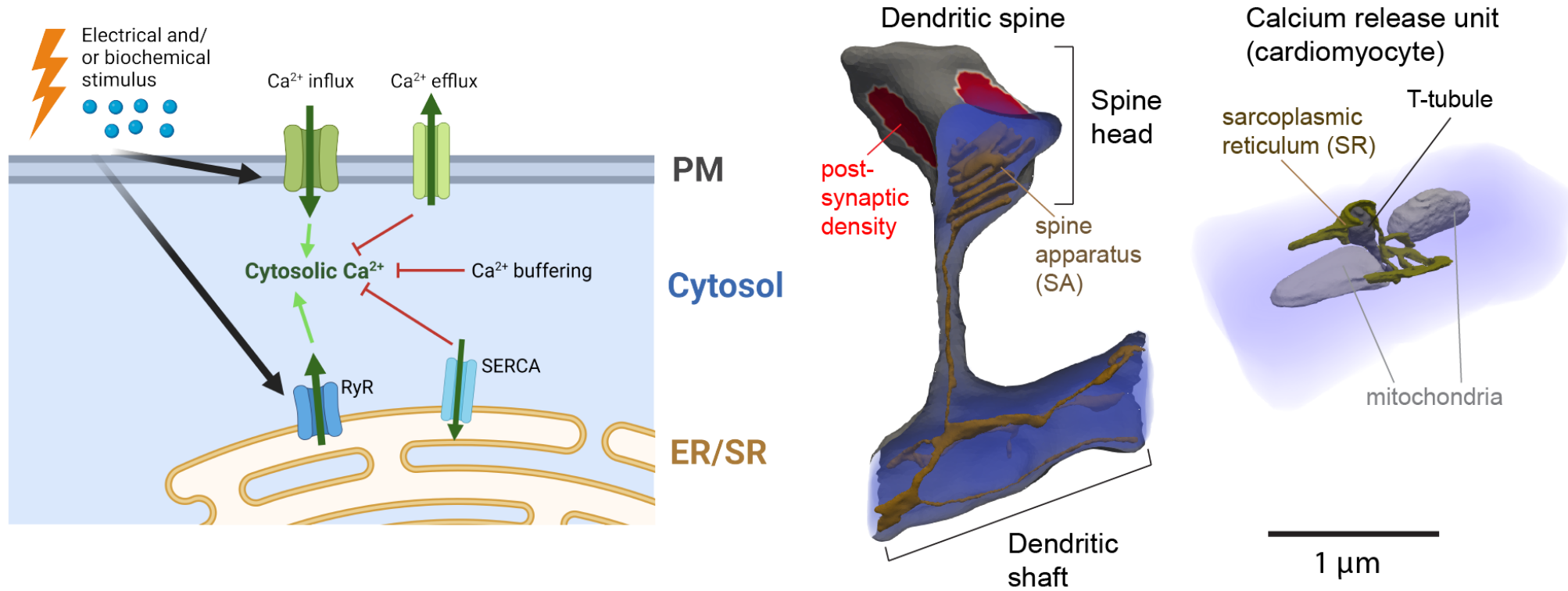
Solution and analysis



	Ω^1		Γ^N		Residual
Ω^1	Ω^1 reactions	...	$\Omega^1 \leftrightarrow \Gamma^N$ reactions	$[u_1^1, u_2^1, \dots]$	
$\vdots$	$\vdots$	$\ddots$		$\vdots$	=
Γ^N	$\Gamma^N \leftrightarrow \Omega^1$ reactions		Γ^N reactions	$[v_1^N, v_2^N, \dots]$	



Test case: examining calcium signaling in realistic geometries



Francis and Laughlin et al 2024, *Nat Comp Sci*

Getting started with SMART

- SMART tutorial repository: <https://github.com/RangamaniLabUCSD/smart-tutorial>
- Given the complicated dependencies and installation of FEniCS, it is generally recommended to run SMART using a containerization software such as Docker or Singularity
- (JupyterHub use instructions omitted in this version of the presentation)

SMART installation instructions via Docker (from smart-tutorial repository)

Using Docker (recommended)

1. Install [Docker Desktop](#) for Windows, Mac, or Linux. In Windows, it is recommended to use Docker with the WSL2 backend, requiring you to first [install WSL2](#) and then follow the [Docker instructions for using the WSL2 backend](#).

2. From your command line (WSL, Mac, or Linux), pull the desired Docker image from the Github registry:

```
docker pull ghcr.io/rangamanilabucsd/smart:latest
```



It is possible to pull a specific version by changing the tag, e.g.

```
docker pull ghcr.io/rangamanilabucsd/smart:v2.0.1
```



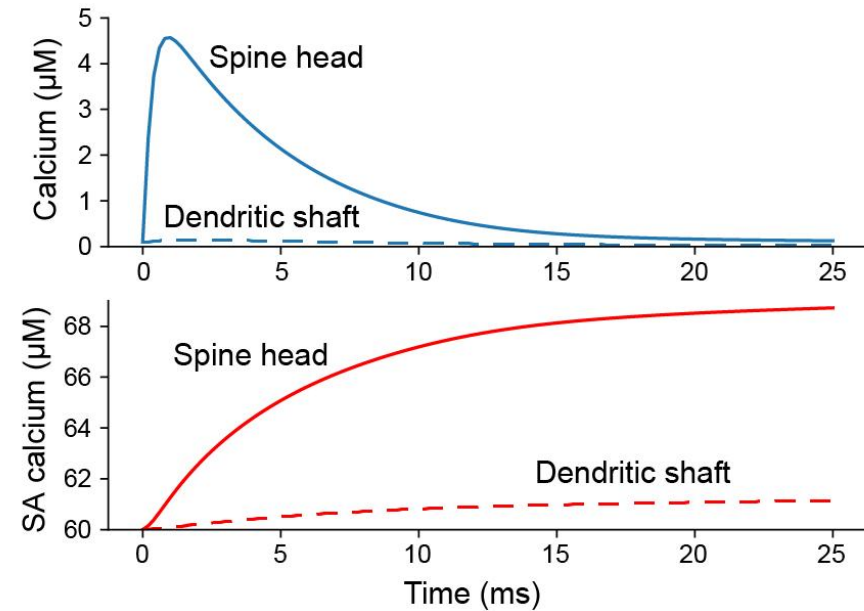
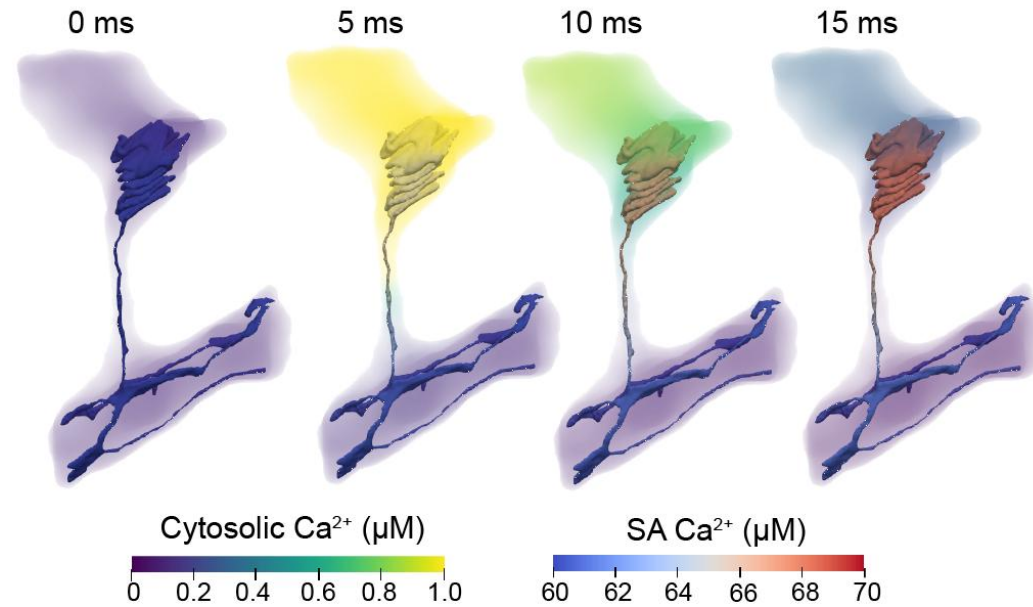
will use version 2.0.1.

3. In order to start a container you can use the [docker run](#) command. For example the command

```
docker run -v $(pwd):/home/shared -w /home/shared -ti --name smart-tutorial ghcr.io/rangamanilabucsd
```



Calcium dynamics in a dendritic spine

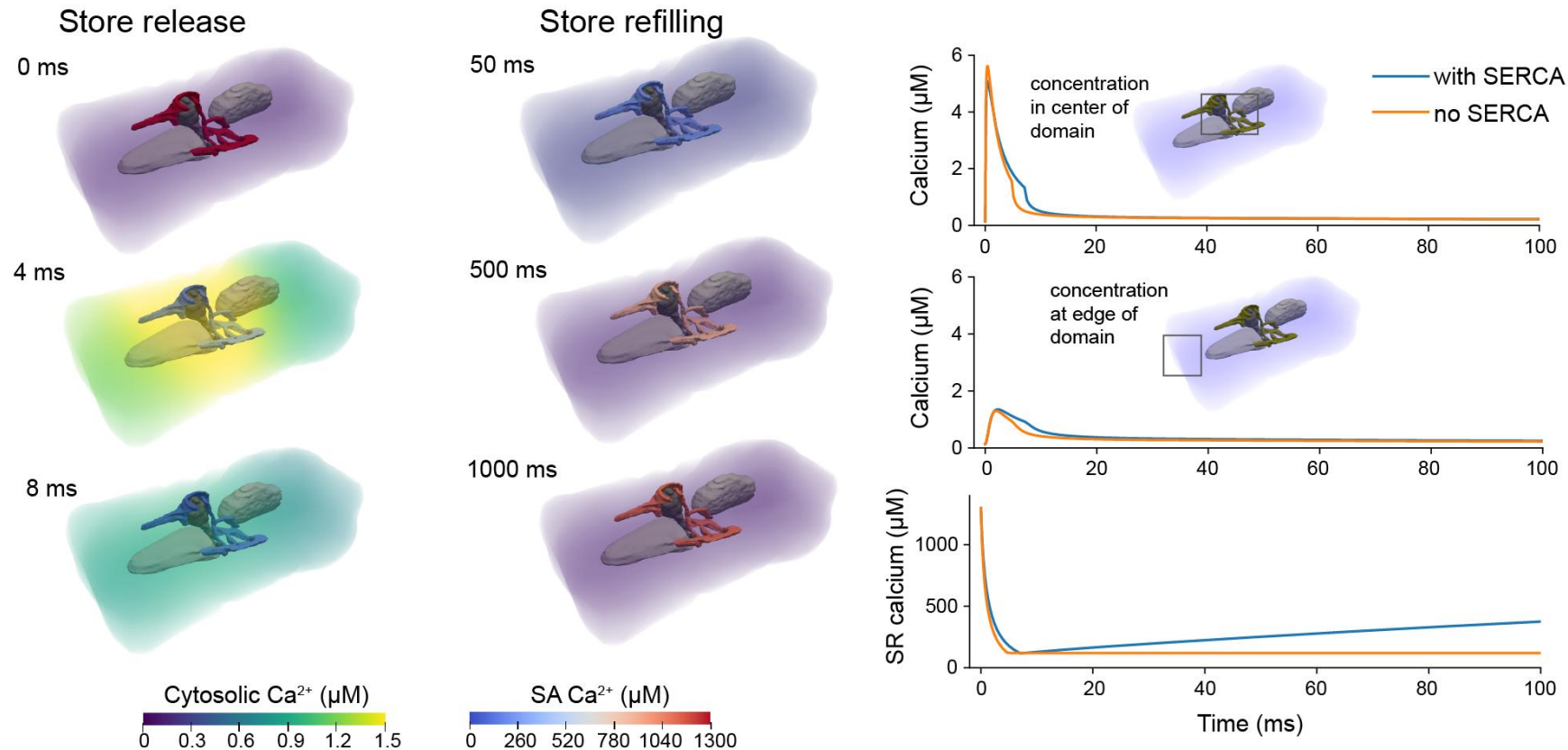


Original implementation of this model by Bell et al 2019, *J Gen Physiology*.

Francis and Laughlin et al 2024, *Nat Comp Sci*

Calcium release unit in cardiomyocyte

- In the absence of store refilling due to SERCA activity, Ca^{2+} release is not as sustained, in agreement with the original implementation of this model by Hake et al 2012, *J Physiology*.



First steps for running this tutorial

1. Login to a JupyterHub (if provided) or run SMART via a Docker container as described in the repository.
2. Open a terminal after logging in and clone the smart-tutorial and (optionally) the SMART repository itself in the scratch directory.

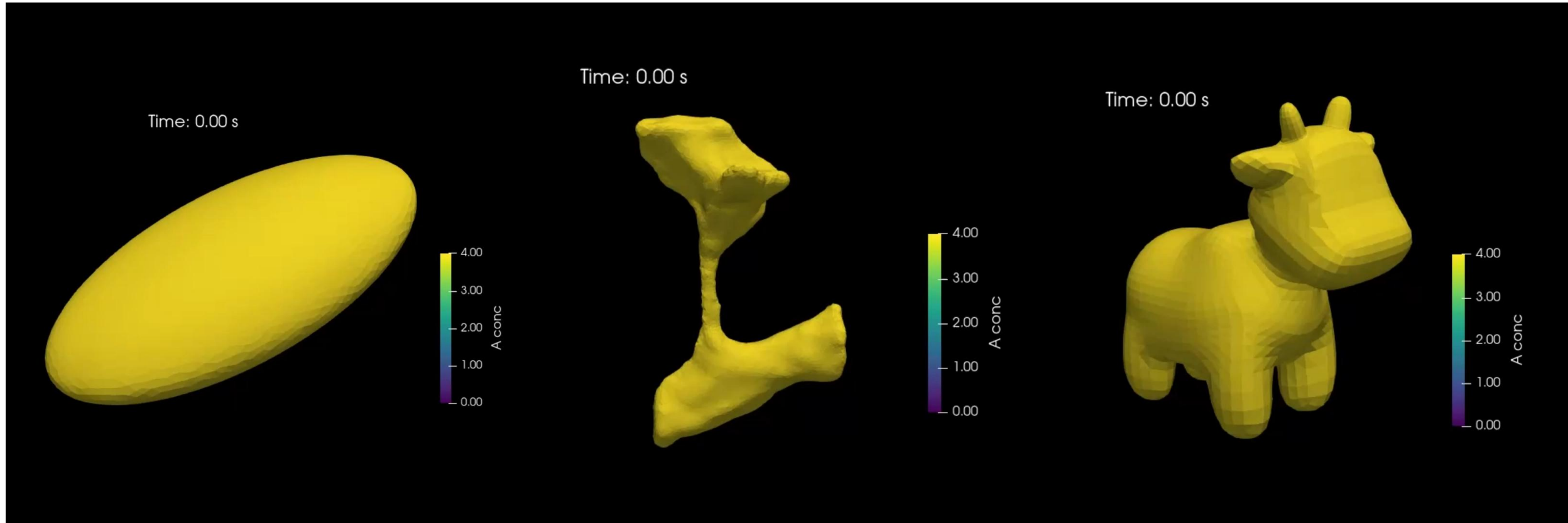
```
cd scratch
```

```
git clone https://github.com/RangamaniLabUCSD/smart-tutorial
```

```
git clone https://github.com/RangamaniLabUCSD/smart
```

3. Open the file `scratch/smart-tutorial/smart_minimal.ipynb`

Videos from initial examples

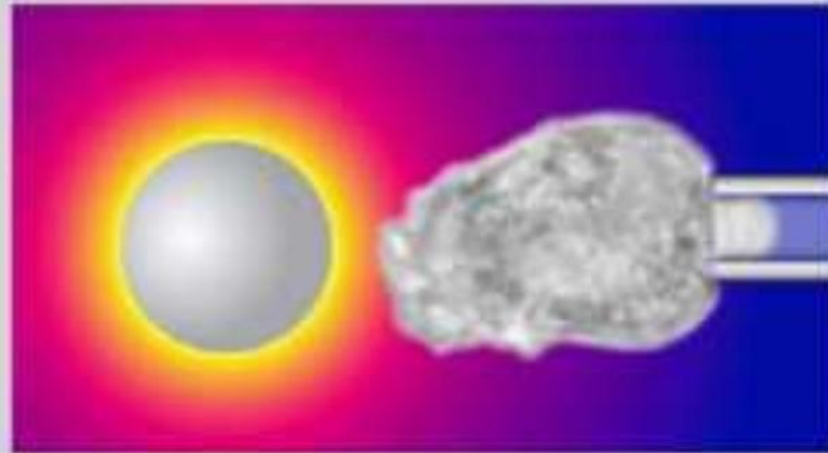


Case study: Reaction-diffusion of chemoattractant

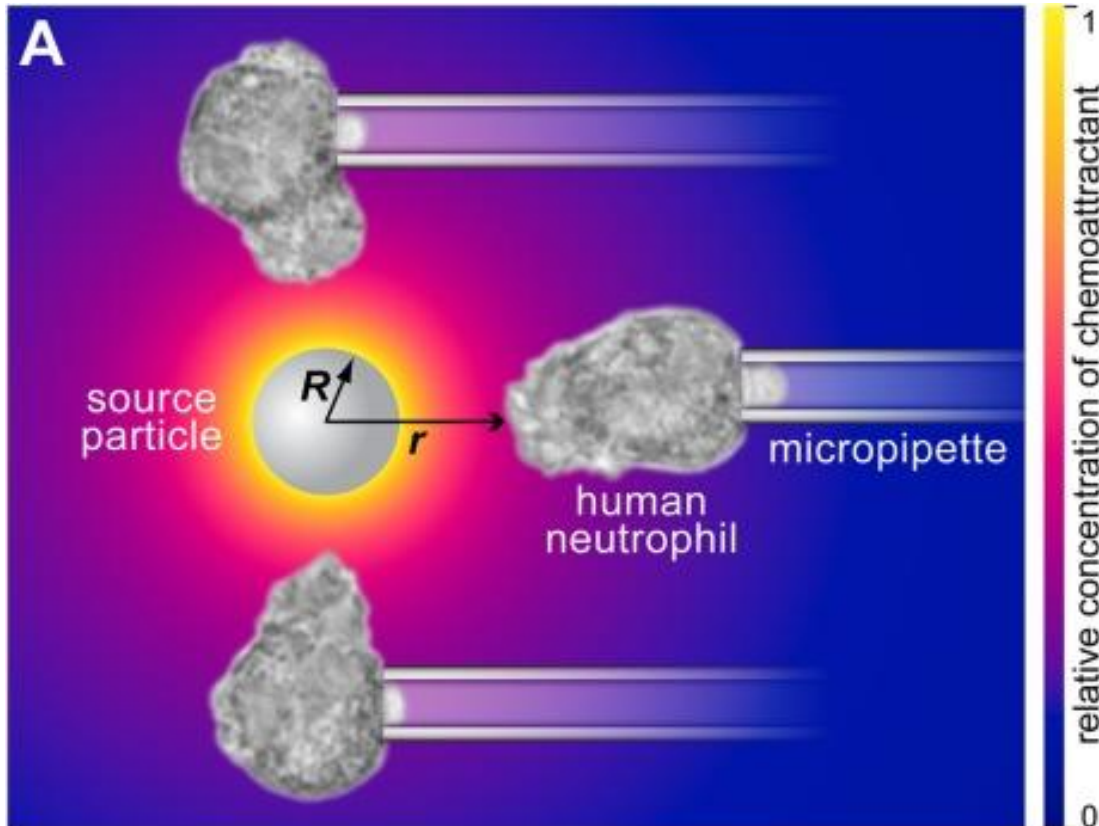
How close do I need to bring a pathogen to an immune cell (neutrophil here) in order for the white blood cell to detect the pathogen location?

(Francis and Heinrich 2017, *Biophys J*, Heinrich et al 2017 *Front. Immunol*)

Immune cells are uniquely capable
(bio)detectors of pathogens.



Case study: Reaction-diffusion of chemoattractant



$$\frac{\partial c}{\partial t} = D \nabla^2 c - kc$$

Initial condition:

$$c = 0$$

BCs:

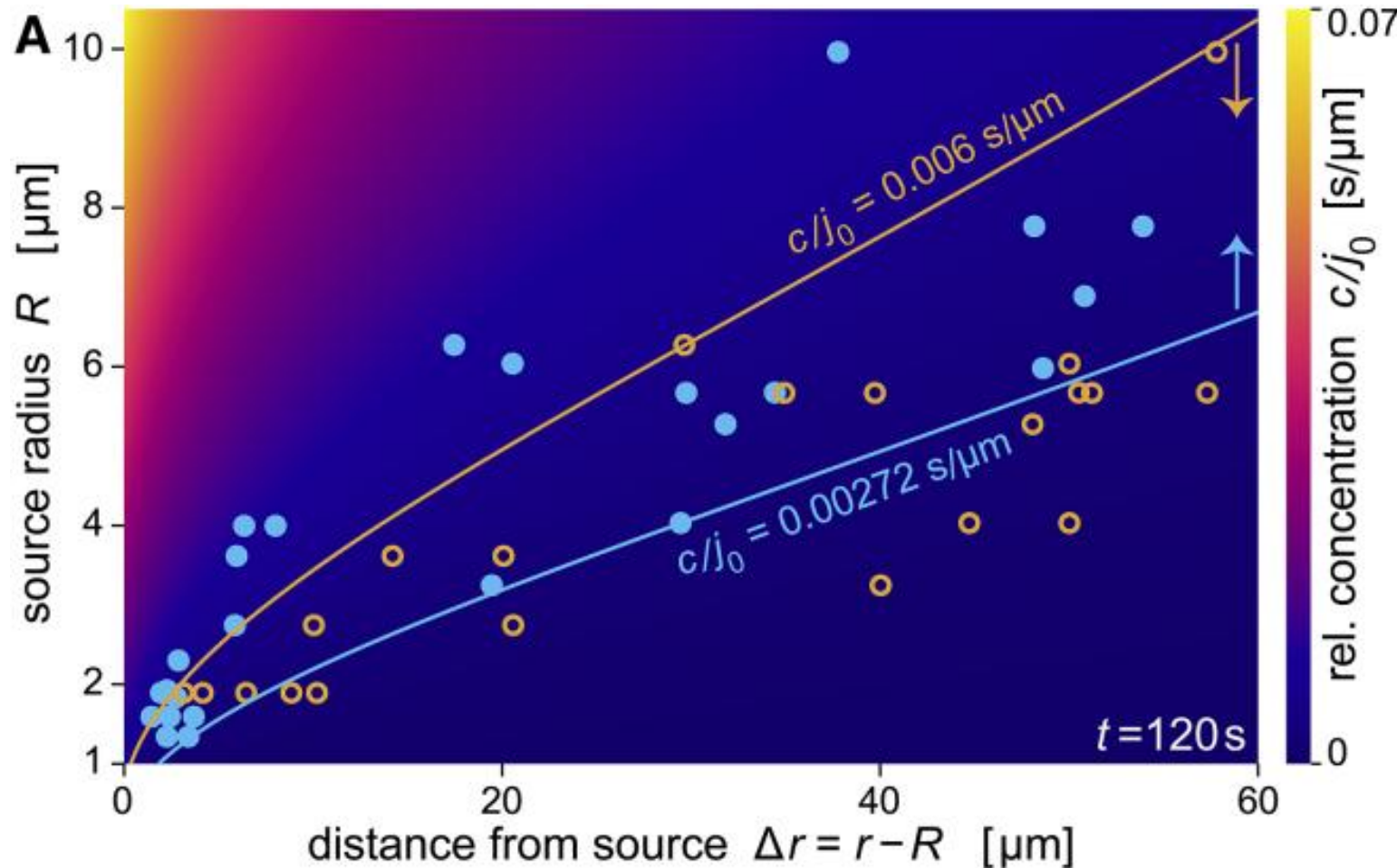
$$c = 0 \text{ for } r \rightarrow \infty,$$

$$\frac{\partial c}{\partial r} = -\frac{j_0}{D} \text{ at } r = R$$

Analytical solution ($\Delta r = r - R$):

$$c(\Delta r, t) = \frac{j_0 R^2}{\Delta r + R} \left\{ \begin{aligned} &\frac{1}{2(D+R\sqrt{Dk})} \exp\left(-\frac{\Delta r}{\sqrt{D/k}}\right) \operatorname{erfc}\left(\frac{\Delta r}{2\sqrt{Dt}} - \sqrt{kt}\right) \\ &+ \frac{1}{2(D-R\sqrt{Dk})} \exp\left(\frac{\Delta r}{\sqrt{D/k}}\right) \operatorname{erfc}\left(\frac{\Delta r}{2\sqrt{Dt}} + \sqrt{kt}\right) \\ &- \frac{1}{D-kR^2} \exp\left[\frac{\Delta r}{R} + \left(\frac{D}{R^2} - k\right)t\right] \operatorname{erfc}\left(\frac{\Delta r}{2\sqrt{Dt}} + \frac{\sqrt{Dt}}{R}\right) \end{aligned} \right\}$$

Case study: Reaction-diffusion of chemoattractant

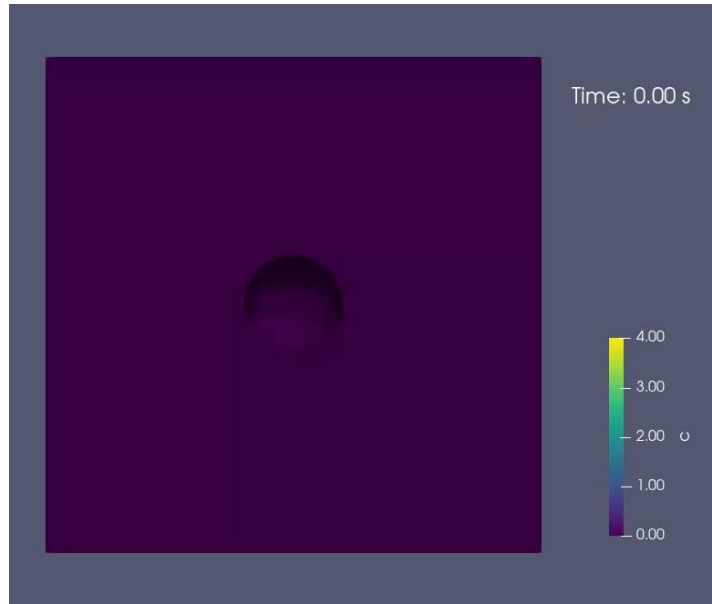


Additional questions we had (now readily addressable in SMART):

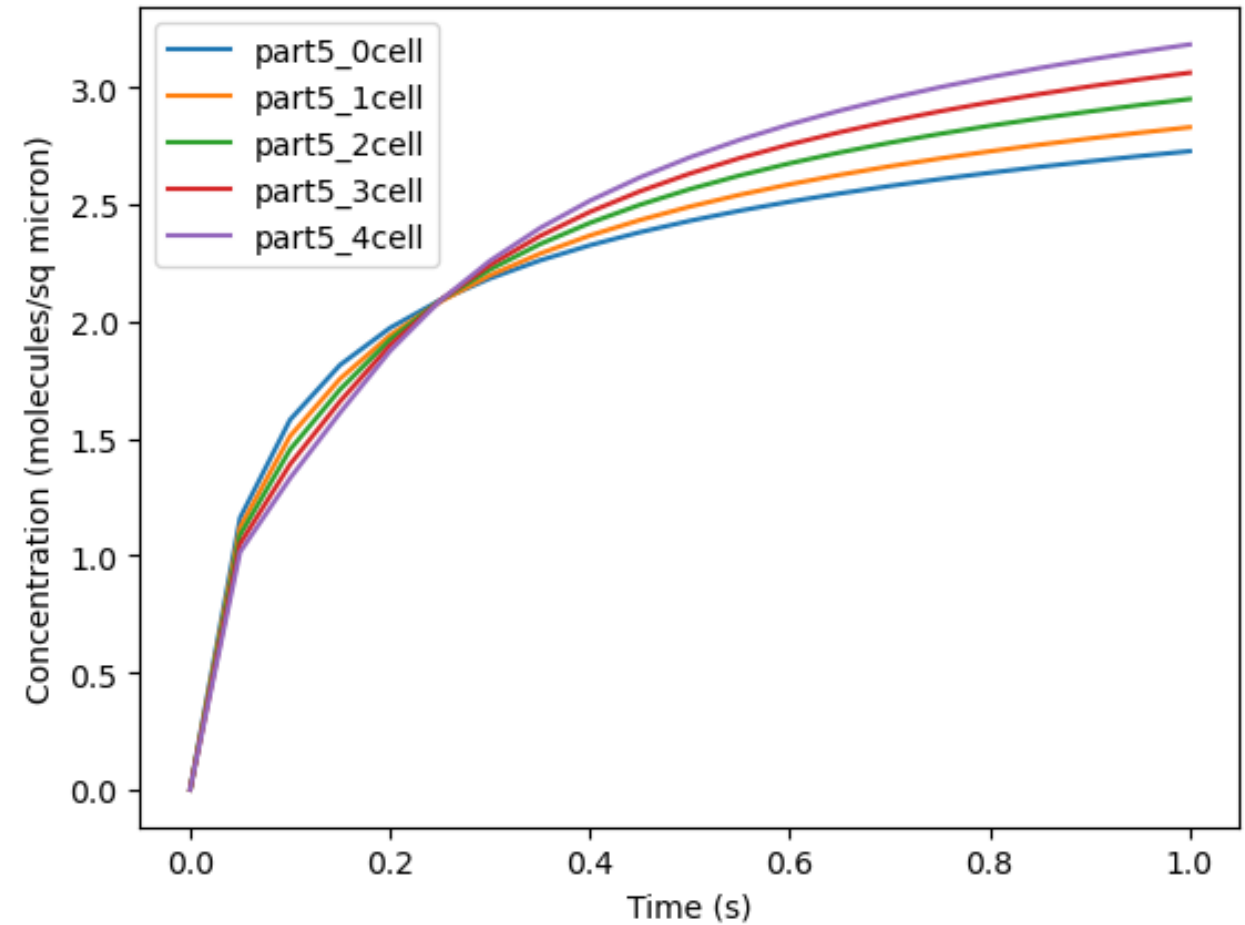
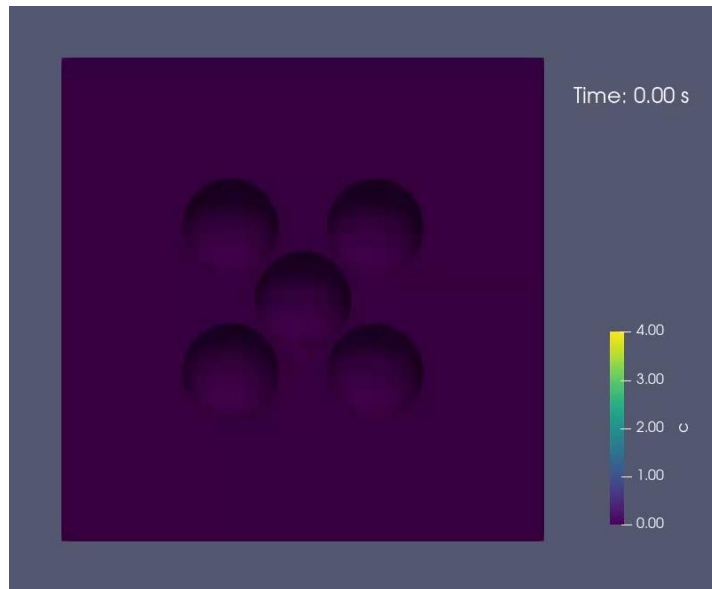
- How does this extend to non-spherical particles?
- What happens if you add fluid flow?
- What happens when you have multiple sources?
- How much does the presence of a cell (or multiple cells) in the domain perturb this solution?

Case study: Reaction-diffusion of chemoattractant

0 cells



4 cells



Case study: Reaction-diffusion of chemoattractant

Questions to test:

1. What are the effects of changing particle size and/or aspect ratio? (use `part5_changeshape.ipynb`)
2. What are the effects of adding advective flow? (use `part5_advection.ipynb`)

```
axisymm = True
AR = SET ASPECT RATIO HERE
sourceRad = SET SOURCE RADIUS HERE
if axisymm:
    z0 = 50.0
    dmesh, facet_markers, cell_markers = mesh_tools.create_2Dcell(outerExpr=f"(r/{z0})",
                                                                    innerExpr=f"(r/({sourceRad}*{AR**(1/3)}))**2",
                                                                    hEdge=5.0, hInnerEdge=0.05, outer_marker=10,
                                                                    outer_tag=1, half_cell=True)
else:
    z0 = 0.0
    dmesh, facet_markers, cell_markers = mesh_tools.create_multicell(cubeSize=50, loc\
                                                                    cellRad1=[sourceRad*AR**(1/3), source\
                                                                    hCube=5.0, hCell=0.5,
                                                                    interface_marker1=11, outer_marker=1\
                                                                    extracell_tag=1)
```

Now we define the compartments, where we can introduce advection, before solving the system. Note that in the axisymmetric case, fluid flow is by definition zero in the second dimension.

```
D_unit = unit.um**2 / unit.s
conc_unit = unit.molecule / unit.um**3
surf_unit = unit.molecule / unit.um**2
if axisymm:
    EC_var = Compartment("EC", 2, unit.um, 1,
                        vel=[SET VELOCITY HERE (ex: [0.0,0.0,50.0] or ["0.0","0.0","50.0*x"])]])
    source_var = Compartment("source", 1, unit.um, 11)
    outer_var = Compartment("outer", 1, unit.um, 10)
else:
    EC_var = Compartment("EC", 3, unit.um, 1,
                        vel=[SET VELOCITY HERE (ex: [0.0,0.0,50.0] or ["0.0","0.0","50.0*x"])]])
    source_var = Compartment("source", 2, unit.um, 11)
    outer_var = Compartment("outer", 2, unit.um, 10)
```

Case study: Reaction-diffusion of chemoattractant

3. How does the presence of multiple sources alter the solution? (use part5_moresources.ipynb)

```
AR = 1.5
radVec = [5.0*AR**(1/3), 5.0*AR**(1/3), 5.0*AR**(-2/3)]
locVec1 = [[0,0,0], DEFINE ADDITIONAL LOCATIONS HERE]
cellRad1 = len(locVec1)*radVec
dmesh, facet_markers, cell_markers = mesh_tools.create_multicell(cubeSize=50, locVec1=locVec1,
                                                                    cellRad1=cellRad1,
                                                                    hCube=5.0, hCell=0.5,
                                                                    interface_marker1=11, outer_marker=10,
                                                                    extracell_tag=1)
```

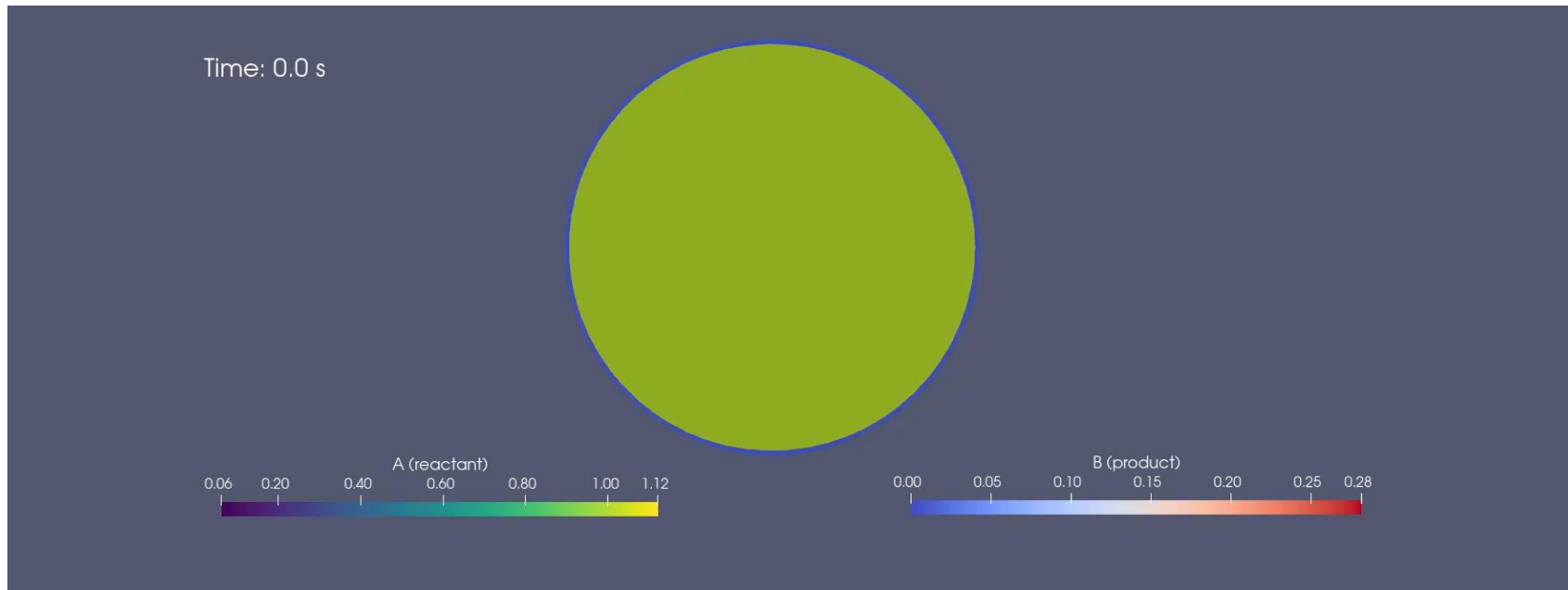
4. How might cell localization disrupt the chemoattractant field at steady-state? (use part5_othercells.ipynb)

```
AR2 = 1.0
otherCellRad = [5.0*AR2**(1/3), 5.0*AR2**(1/3), 5.0*AR2**(-2/3)]
otherCellLoc = [LIST OF OTHER CELL LOCS HERE]
otherCellRad = len(otherCellLoc)*otherCellRad

dmesh, facet_markers, cell_markers = mesh_tools.create_multicell(
    cubeSize=50,
    locVec1=sourceLoc, cellRad1=sourceRad,
    locVec2=otherCellLoc, cellRad2=otherCellRad,
    hCube=5.0, hCell=0.5,
    interface_marker1=11, interface_marker2=12, ou
    extracell_tag=1)
```

Current development in SMART

1. Including advective transport in both Lagrangian and Eulerian formulations (feature currently introducing a pre-release as shown today).
2. Developing new examples including reaction, advection, and diffusion within meshes including multiple cells.
3. Allow for well-mixed subregions of the geometry (0D compartments).
4. Features to support parameter estimation.
5. Migration to FEniCSx.



Acknowledgements

The Rangamani Lab:

- **Prof. Padmini Rangamani**
- **Emmet Francis**
- Patrick Noerr
- Kyle Stark
- Ji Yeon Hyun
- Daniel Mansour
- Anooshirvan Mahdavian
- Aanya Sawhney

Past members involved with the work shown today:

- **Justin Laughlin**
- **Chris Lee**

Simula Research

Laboratory collaborators:

- **Marie Rognes**
- **Jørgen Dokken**
- **Henrik Finsberg**

simula

Funding:

- Multi-University Research Initiative (MURI) - funding from the U.S. Air Force
- Wu Tsai Human Performance Alliance
- This material is based upon work supported by the National Science Foundation under Grant # EEC-2127509 to the American Society for Engineering Education.

